
Chapter 7: Entity-Relationship Model



Database Design and E-R Model

- ▶ Up to this point, we have *assumed a given database schema* and studied how queries and updates are expressed.
- ▶ We now consider **how to design a database schema in the first place**.
- ▶ In this chapter, we focus on the **entity-relationship data model**, which provides *a means of identifying entities to be represented in the database and how those entities are related*.



Outline

- ▶ **Design Process**
- ▶ **Modeling**
- ▶ **Constraints**
- ▶ **E-R Diagram**
- ▶ **Design Issues**
- ▶ **Weak Entity Sets**
- ▶ **Extended E-R Features**
- ▶ **Reduction to Relation Schemas**
- ▶ **Database Design**
- ▶ **UML**



Overview of the Design Process

- ▶ The task of **creating a database application** is complex
 - ▶ Design of the *database schema*
 - ▶ Design of the *programs* that access and update the data
 - ▶ Design of a *security scheme* to control access to data
- ▶ Often no one person understands the **complete data needs** of an application.
- ▶ The *database designer* must interact with *users of the application* to understand the needs of the application, represent them in a *high-level fashion* that can be understood by the users, and then translate the requirements into *lower levels of the design*.



Design Phases

- ▶ The **phases of database design**:
 - ▶ Initial phase of database design is to characterize fully the *data needs* of the prospective database users and the outcome is a *specification of user requirements*.
 - ▶ Next, the designer chooses a *data model* and, by applying the concepts of the chosen data model, translates these requirements into a *conceptual schema of the database*.
 - ▶ The **entity-relationship model** is typically used to represent the conceptual design.
 - ▶ A fully developed *conceptual schema* also indicates the *functional requirements* of the enterprise (e.g. operations such as modifying or updating data).



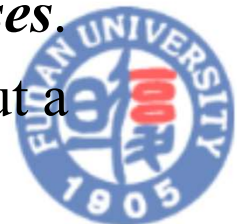
Design Phases (Cont.)

- ▶ **Two final design phases**
 - ▶ **Logical-design phase:** maps the *high-level conceptual schema* onto the *implementation data model* of the database system
 - ▶ **Physical-design phase:** the *physical features* of the database are specified, e.g. the form of file organization and choice of index structures.



Design Alternatives

- ▶ Pitfall 1 in designing a DB schema: **Redundancy**
 - ▶ A bad design may **repeat information**.
 - ▶ If we store the *course identifier* and *title of a course* with each *course offering*, the title would be stored redundantly.
 - ▶ Redundancy can also occur in a *relational schema*
 - ▶ The biggest problem with redundancy is that the *copies of a piece of information can become inconsistent*.
- ▶ Pitfall 2 in designing a DB schema: **Incompleteness**
 - ▶ A bad design may make certain aspects of the enterprise **difficult or impossible to model**.
 - ▶ Suppose that: we only had entities corresponding to *course offering*, without having an entity corresponding to *courses*.
 - ▶ It would then be impossible to represent information about a *new course*, unless that course is offered.



The Entity-Relationship Model

- ▶ The **Entity-Relationship (E-R)** data model was developed to facilitate database design by allowing *specification of an enterprise schema that represents the overall logical structure of a database.*
- ▶ The E-R data model employs three basic concepts:
 - ▶ **Entity sets**
 - ▶ **Relationship sets**
 - ▶ **Attributes**



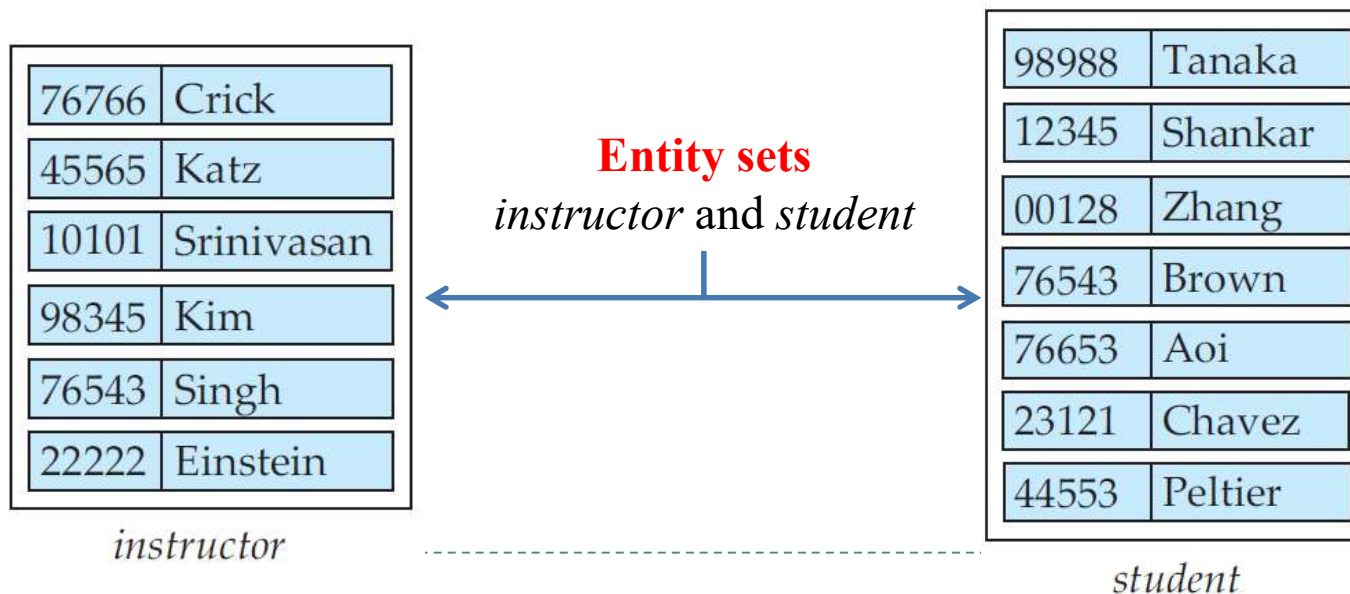
Entity Sets

- ▶ A **database** can be modeled as
 - ▶ *a collection of entities*
 - ▶ *relationship among entities*
- ▶ An **entity** is a “*thing*” or “*object*” in the real world that is distinguishable from other objects.
 - ▶ Example: specific person, company, event, plant
- ▶ Entities have **attributes**
 - ▶ Example: people have *names* and *addresses*



Entity Sets

- ▶ Each entity has a **value** for each of its **attributes**
 - ▶ Example, a particular instructor entity may have the value 12121 for *ID*, the value Wu for *name*
- ▶ An **entity set** is a set of entities of the *same type* that share the same properties.
 - ▶ Example: set of all persons, companies, trees, holidays



Relationship Sets

- ▶ A **relationship** is an association among several *entities*

Example:

44553 (Peltier)	<u>advisor</u>	22222 (<u>Einstein</u>)
<i>student</i> entity	relationship set	<i>instructor</i> entity

- ▶ A **relationship set** is a mathematical relation among $n \geq 2$ entities, each taken from entity sets

$$\{(e_1, e_2, \dots, e_n) \mid e_1 \in E_1, e_2 \in E_2, \dots, e_n \in E_n\}$$

where $(e_1, e_2, \dots, e_n)$ is a relationship

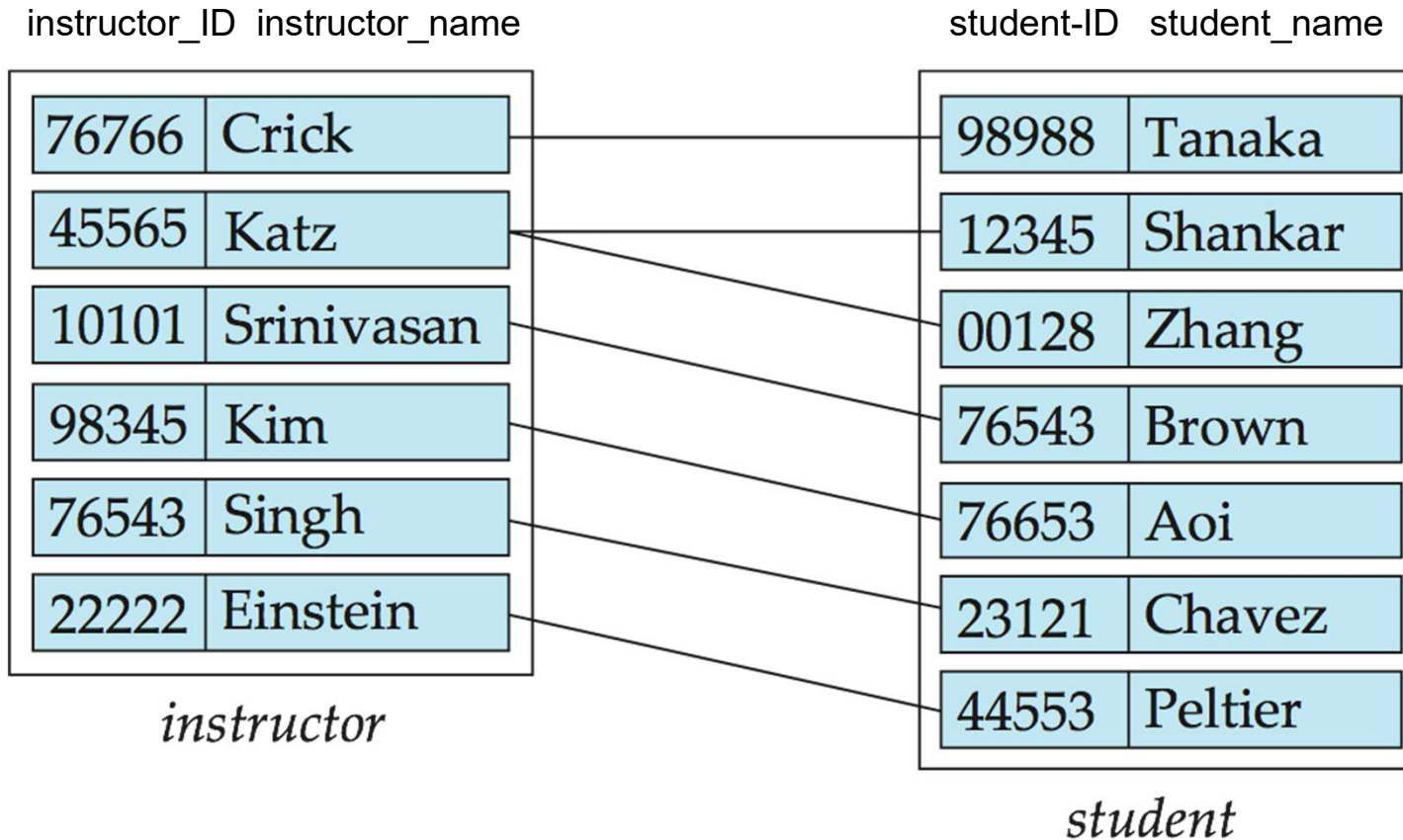
- ▶ Example:

$$(44553, 22222) \in \text{advisor}$$

- ▶ The *function* that an entity plays in a relationship is called that **entity's role**.

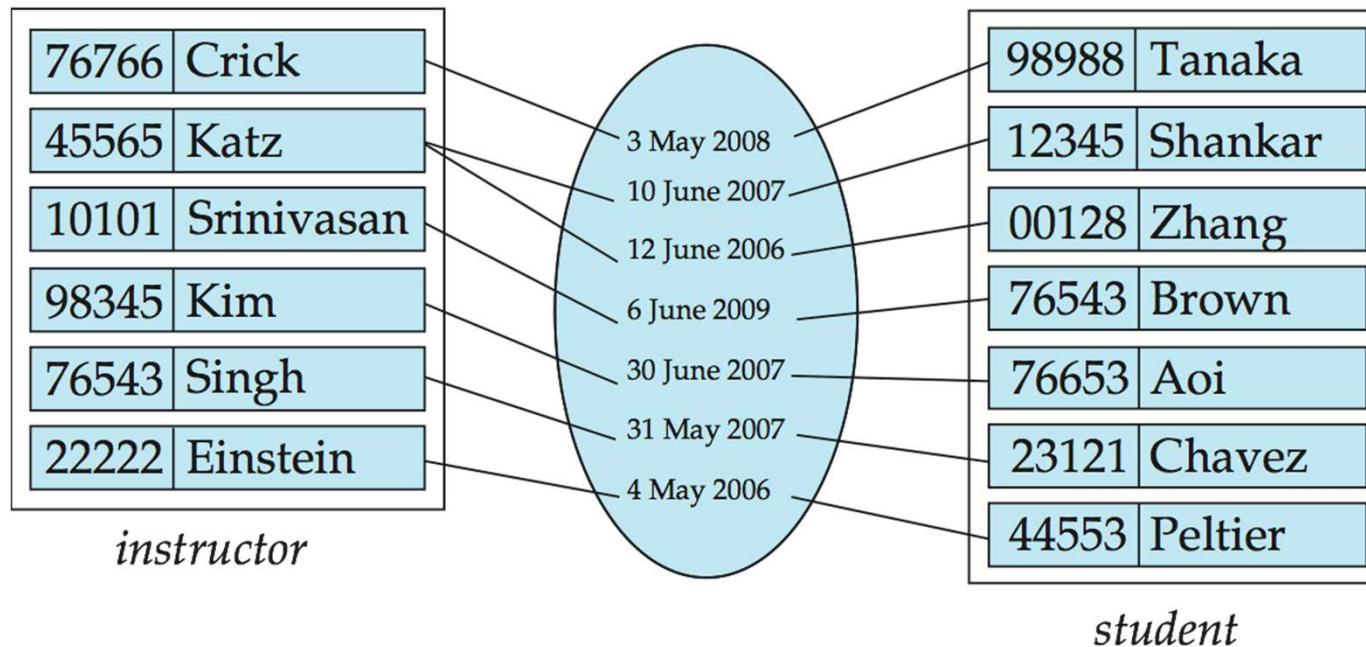


Relationship Set *advisor*



Relationship Sets (Cont.)

- ▶ An **attribute** can also be property of a *relationship set*
 - ▶ For instance, the *advisor* relationship set between entity sets *instructor* and *student* may have the attribute *date* which tracks when the student started being associated with the advisor



Relationship Sets (Cont.)

- ▶ It is possible to have **more than one relationship set** involving *the same entity sets*
 - ▶ In our example, the *instructor* and *student* entity sets participate in the relationship set *advisor*.
 - ▶ Additionally, suppose each *student* must have another *instructor* who serves as a *department advisor* (undergraduate or graduate). Then the *instructor* and *student* entity sets may participate in another relationship set, *dept_advisor*.



Degree of a Relationship Set

- ▶ **Binary relationship**

- ▶ involve two entity sets (or *degree two*).
 - ▶ most relationship sets in a database system are binary.

- ▶ Relationships between more than two entity sets are rare.

- ▶ Three entity sets: *instructor*, *student*, and *project*
 - ▶ Students work on research projects under the guidance of an instructor. To represent this information, we relate the three entity sets through the relationship set *proj_guide*.
 - ▶ Relationship *proj_guide* is a **ternary**(三元的) relationship between instructor, student, and project

- ▶ The number of entity sets that participate in a relationship set is the *degree of the relationship set*.



Attributes

- ▶ An *entity* is represented by **a set of attributes** (descriptive properties possessed by all members of an entity set).
 - ▶ Example: *instructor* = (*ID*, *name*, *street*, *city*, *salary*)
course = (*course_id*, *title*, *credits*)
- ▶ **Domain** (or **Value Set**) – the set of permitted values for each attribute
 - ▶ The domain of attribute *course_id* might be the set of all text strings of a certain length. Similarly, the domain of attribute *semester* might be strings from the set {*Fall*, *Winter*, *Spring*, *Summer*}.



Attributes (Cont.)

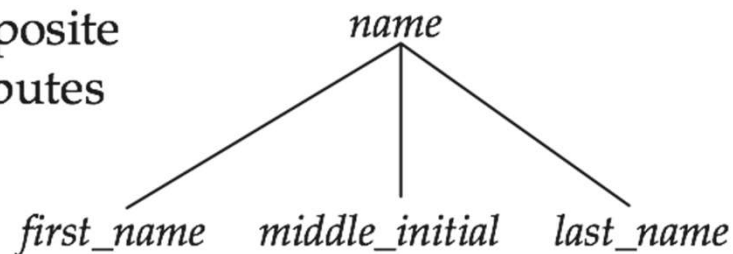
- ▶ Formally, an attribute of an entity set is a **function** that *maps from the entity set into a domain*.
- ▶ An entity set may have several attributes.
- ▶ Each entity can be described by a set of **(attribute, data value)** pairs.
- ▶ Example: a particular *instructor* entity may be described by the set: $\{(ID, 76766), (name, Crick), (dept_name, Biology), (salary, 72000)\}$



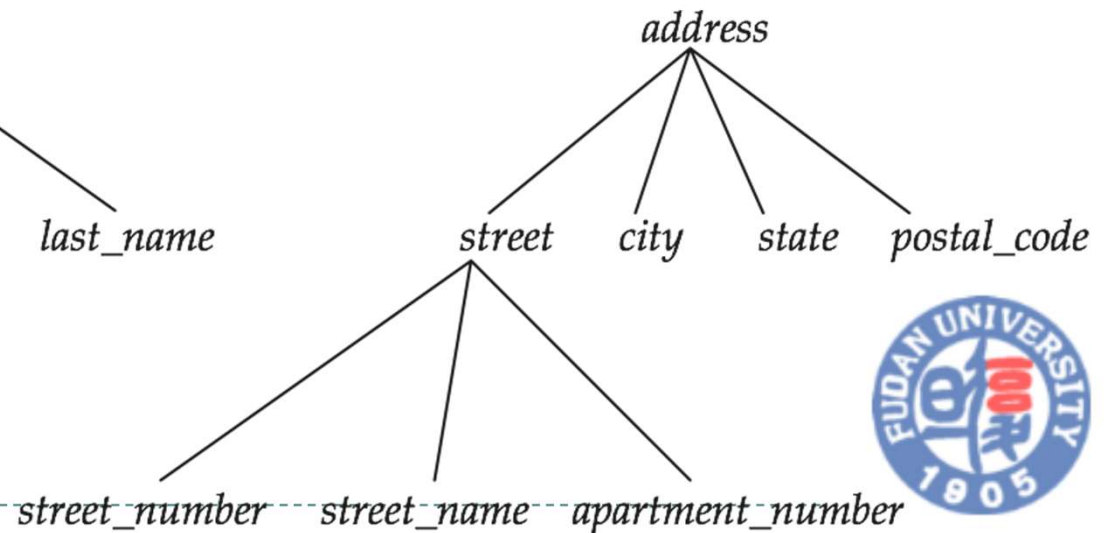
Attribute types

- ▶ **Simple** and **composite** attributes
 - ▶ *Simple* attributes have not been divided into subparts.
 - ▶ *Composite* attributes can be divided into subparts
 - ▶ For example, an attribute *name* could be structured as a composite attribute consisting of *first_name*, *middle_initial*, and *last_name*.

composite
attributes



component
attributes



Attribute Type (Cont.)

- ▶ **Single-valued** and **multivalued** attributes
 - ▶ Example: *single-values* attributes: *student_ID*
 - ▶ An *instructor* may have *zero, one, or several* *phone numbers*, and different *instructors* may have different numbers of *phones*.
 - ▶ Example: *multivalued* attribute: *phone_numbers*
 - ▶ To denote that an attribute is multivalued, we enclose it in braces, for example *{phone number}*



Attribute Type (Cont.)

- ▶ **Derived** attributes

- ▶ The value for this type of attribute can be *derived from the values of other related attributes or entities*.
- ▶ Example: *age*, given *date_of_birth*. we can calculate *age* from *date_of_birth* and the current date.
- ▶ Thus, *age* is a derived attribute. In this case, *date_of_birth* may be referred to as a **base attribute/stored attribute**. The value of a derived attribute is not stored but is computed when required.



Attribute - Null Value

- ▶ An attribute takes **a null value** when an entity does not have a value for it.
- ▶ The *null* value may indicate “*not applicable*”
 - ▶ For example, one may have no middle name.
- ▶ *Null* can also designate that an attribute value is *unknown*.
 - ▶ **missing**: the value does exist, but we do not have that information
 - ▶ **not known**: we do not know whether or not the value actually exists



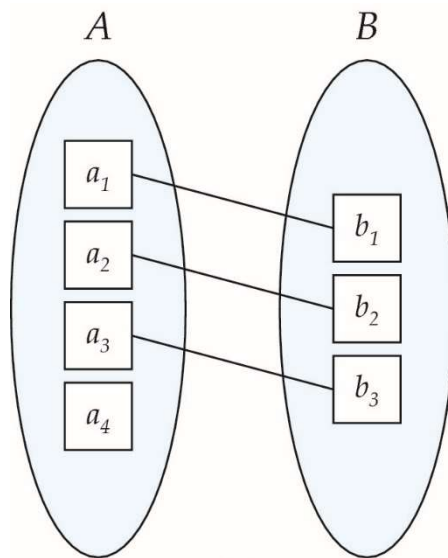
Constraints - Mapping Cardinality

- ▶ Express the **number of entities** to which *another entity* can be associated via a *relationship set*.
- ▶ Most useful in describing binary relationship sets.
- ▶ For a **binary relationship set** the mapping cardinality must be one of the following types:
 - ▶ *One to one*
 - ▶ *One to many*
 - ▶ *Many to one*
 - ▶ *Many to many*



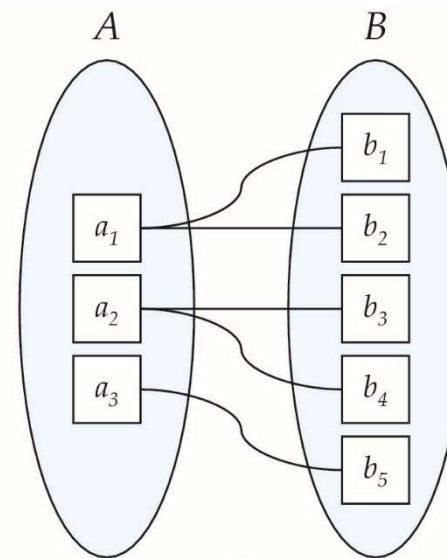
Mapping Cardinalities

- ▶ **One-to-One.** An entity in A is associated with *at most one* entity in B , and an entity in B is associated with *at most one* entity in A .
- ▶ **One-to-Many.** An entity in A is associated with *any number (zero or more)* of entities in B . An entity in B , however, can be associated with *at most one* entity in A .



(a)

One to one

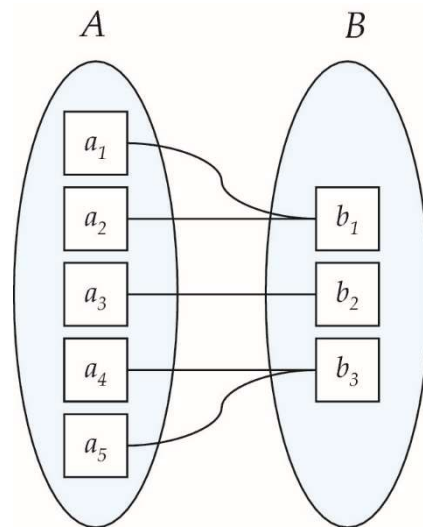


(b)

One to many

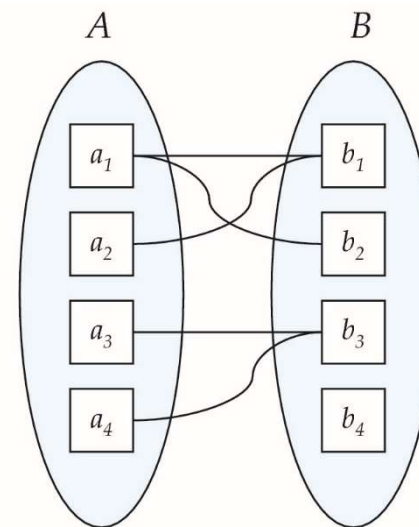
Mapping Cardinalities

- ▶ **Many-to-one.** An entity in A is associated with *at most one entity* in B . An entity in B , however, can be associated with *any number (zero or more)* of entities in A .
- ▶ **Many-to-many.** An entity in A is associated with *any number (zero or more)* of entities in B , and an entity in B is associated with *any number (zero or more)* of entities in A .



(a)

Many to one



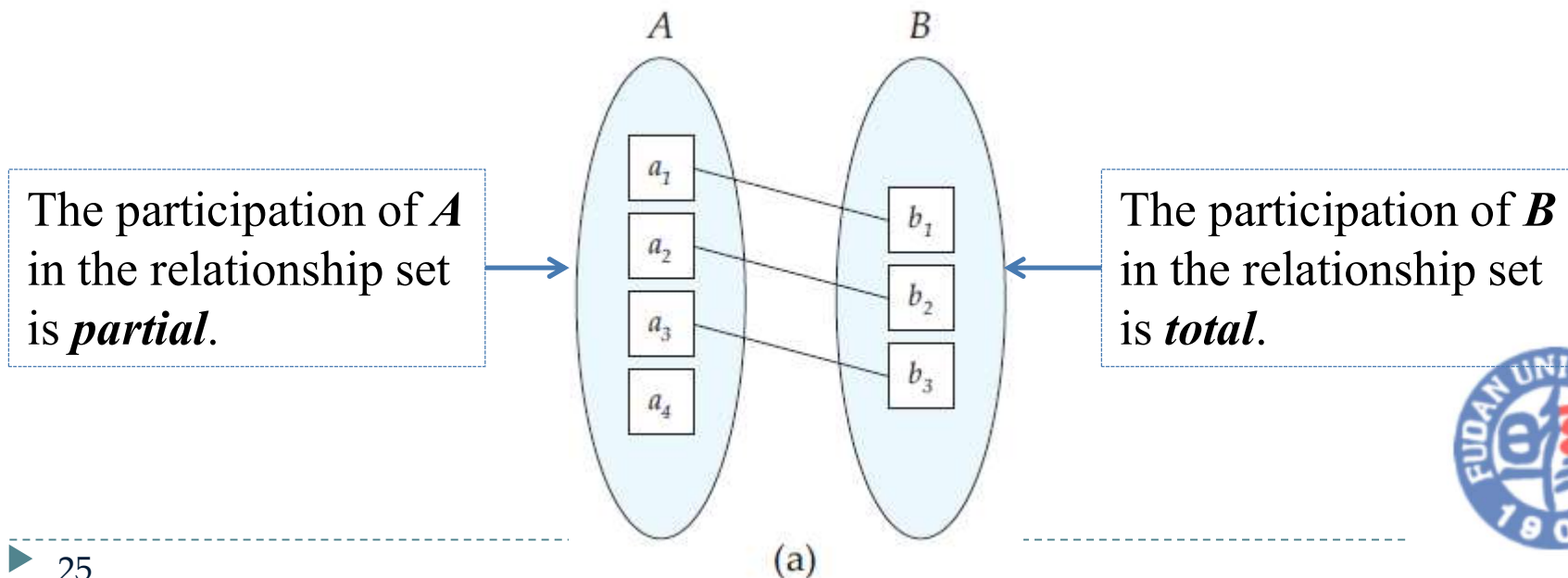
(b)

Many to many



Constraints - Participation Constraints

- ▶ The **participation** of an entity set E in a relationship set R is said to be **total** if *every entity in E participates in at least one relationship in R* .
- ▶ If only some entities in E participate in relationships in R , the participation of entity set E in relationship R is said to be **partial**.



Constraints - Participation Constraints

▶ Example

- ▶ For example, we expect **every** *student* entity to be related to **at least one** *instructor* through the *advisor* relationship. Therefore the participation of *student* in the relationship set *advisor* is **total**.
- ▶ In contrast, an *instructor* need not advise any *students*. Hence, the participation of *instructor* in the *advisor* relationship set is therefore **partial**.



Constraints - Keys

- ▶ A **super key** of an entity set is a set of one or more attributes whose values uniquely determine each entity.
- ▶ A **candidate key** of an entity set is a minimal super key
 - ▶ *ID* is candidate key of *instructor*
 - ▶ *course_id* is candidate key of *course*
- ▶ Although several candidate keys may exist, one of the candidate keys is selected to be the **primary key**.



Keys for Relationship Sets

- ▶ The **primary key** of an entity set allows us to distinguish among the various **entities** of the set.
- ▶ We need a **similar mechanism** to *distinguish among the various relationships of a relationship set*.
- ▶ Let R be a relationship set involving entity sets $E_1, E_2, \dots, E_n$; **Primary-key (E_i)** denote the set of attributes that forms the primary key for entity set E_i .
- ▶ The **composition of the primary key** for a **relationship set** depends on the set of attributes associated with the relationship set R .



Keys for Relationship Sets (Cont.)

- ▶ Relationship set R has **no attributes** associated with it:

$$\text{primary-key}(E_1) \cup \text{primary-key}(E_2) \cup \dots \cup \text{primary-key}(E_n)$$

(describing an individual relationship in set R)

- ▶ Relationship set R has **attributes** $a_1, a_2, \dots, a_m$ associated with it:

$$\text{primary-key}(E_1) \cup \text{primary-key}(E_2) \cup \dots \cup \text{primary-key}(E_n)$$

$$\cup \{a_1, a_2, \dots, a_m\}$$

(describing an individual relationship in set R)

- ▶ In both of the above cases, the set of attributes:

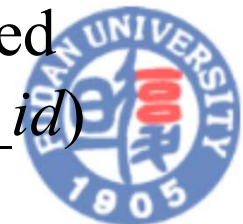
$$\text{primary-key}(E_1) \cup \text{primary-key}(E_2) \cup \dots \cup \text{primary-key}(E_n)$$

forms a **superkey** for the relationship set R



Keys for Relationship Sets (Cont.)

- ▶ **Rename the attributes:** if the attribute names of primary keys are *not unique* across entity sets.
 - ▶ The name of the entity set combined with the name of the attribute would form a unique name.
 - ▶ E.g.: (*student_id*, *instructor_id*) is the super key of *advisor*
- ▶ **Note:** If an entity set participates *more than once* in a relationship set, the *role name* is used instead of the name of the entity set, to *form a unique attribute name*.
 - ▶ E.g.: the entity set *course*, the relation set *prereq*
 - ▶ one *course* (C_2) is a **prerequisite** for another *course* (C_1)
 - ▶ To form a unique attribute name, the role name is used instead of the name of entity set: (*prereq_id*, *course_id*)



Keys for Relationship Sets (Cont.)

- ▶ Consider the **mapping cardinality** of the relationship set when deciding *what are the candidate keys*
 - ▶ Consider the entity sets *instructor* and *student*, and the relationship set *advisor*, with attribute *date*.
 - ▶ Suppose that the relationship set is **many-to-many**. Then the primary key of *advisor* consists of the *union of the primary keys* of *instructor* and *student*.
 - ▶ If the relationship is **many-to-one** from student to instructor — that is, each student can have at most one advisor — then the primary key of *advisor* is simply the primary key of student.
- ▶ Need to consider *semantics of relationship set* in selecting the primary key in case of more than one candidate key.



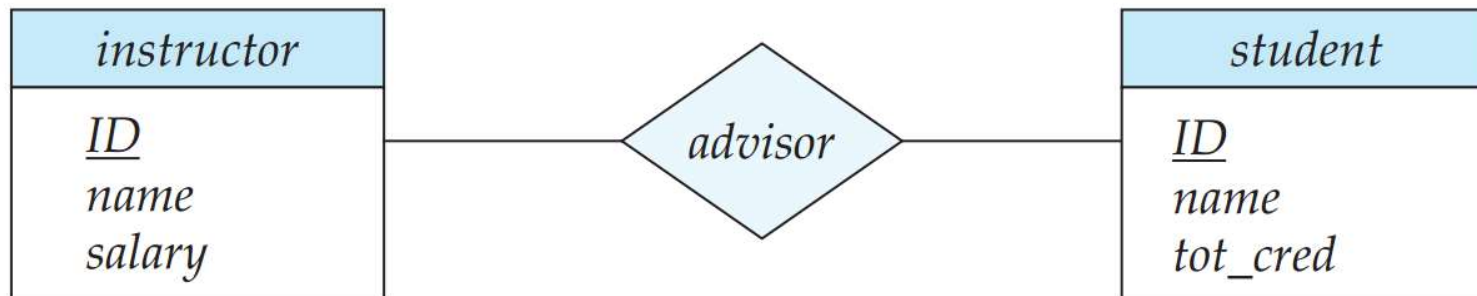
Redundant Attributes

- ▶ Suppose we have **entity sets**
 - ▶ *instructor* (with attributes including *dept_name*)
 - ▶ *department*
- ▶ and a **relationship**
 - ▶ *inst_dept* relating *instructor* and *department*
- ▶ Attribute *dept_name* in entity *instructor* is redundant since there is an explicit relationship *inst_dept* which relates *instructor* to *department*
 - ▶ The attribute replicates information present in the relationship, and should be removed from *instructor*
 - ▶ **BUT**: when *converting back to tables*, in some cases the attribute gets **reintroduced**, as we will see.



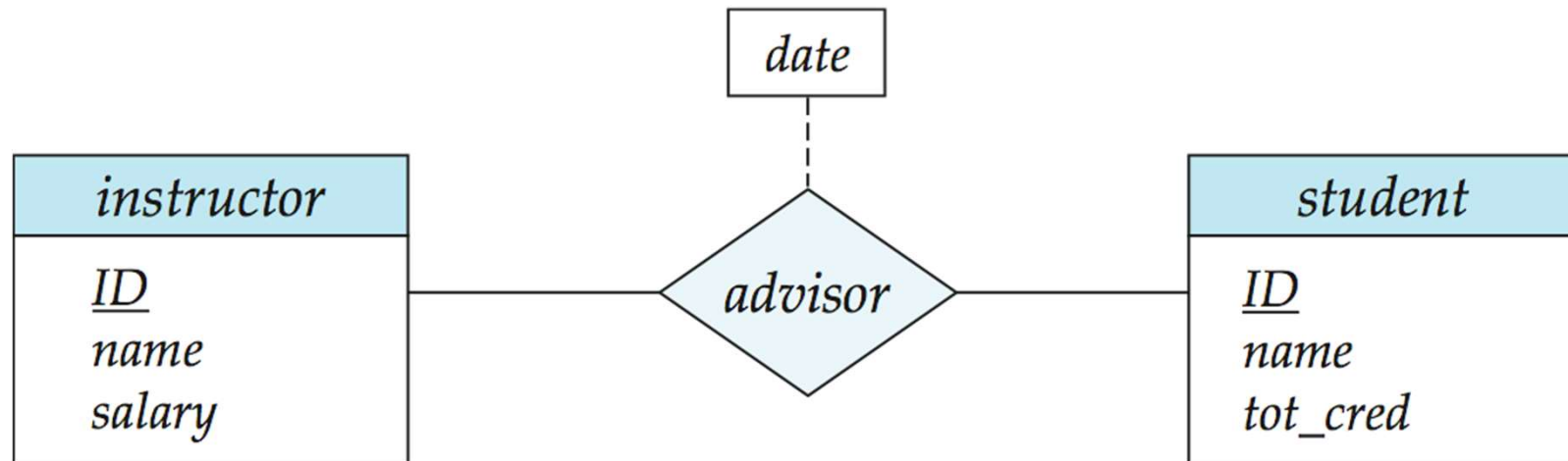
E-R Diagrams

- ▶ An **E-R diagram** can express the *overall logical structure of a database graphically*.
- ▶ An E-R diagram consists of the following major components:
 - ▶ **Rectangles** represent *entity sets*
 - ▶ **Diamonds** represent *relationship sets*
 - ▶ **Attributes** listed inside *entity rectangle*
 - ▶ **Underline** indicates *primary key attributes*
 - ▶ **Lines** link *entity sets* to *relationship sets*



Relationship Sets with Attributes

- ▶ If a **relationship set** has some attributes associated with it, then we enclose the attributes in a *rectangle* and link the rectangle with a *dashed line* to the diamond representing that relationship set.



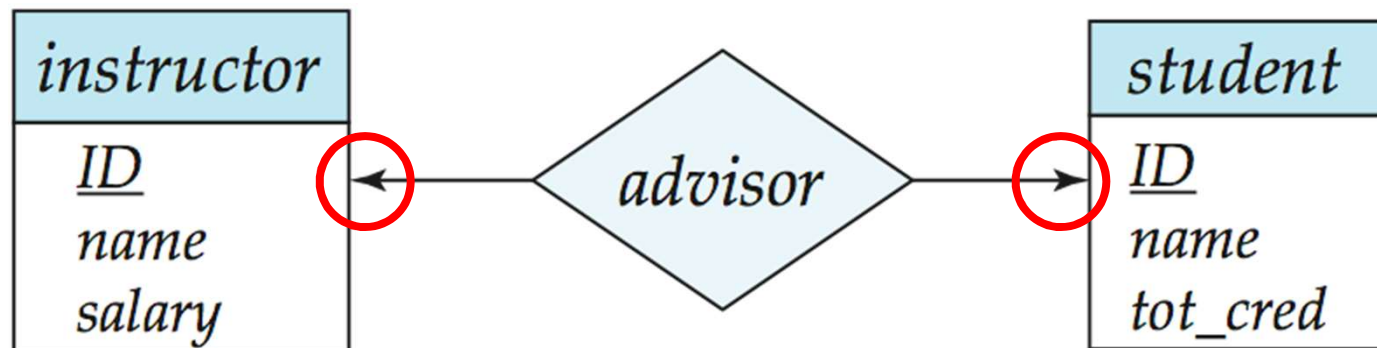
Cardinality Constraints

- ▶ We express **cardinality constraints** by drawing either a *directed line* ($\rightarrow$), signifying “*one*,” or an *undirected line* (—), signifying “*many*”, between the relationship set and the entity set.
- ▶ **One-to-one** relationship
- ▶ **One-to-many** relationship
- ▶ **Many-to-one** relationship
- ▶ **Many-to-many** relationship



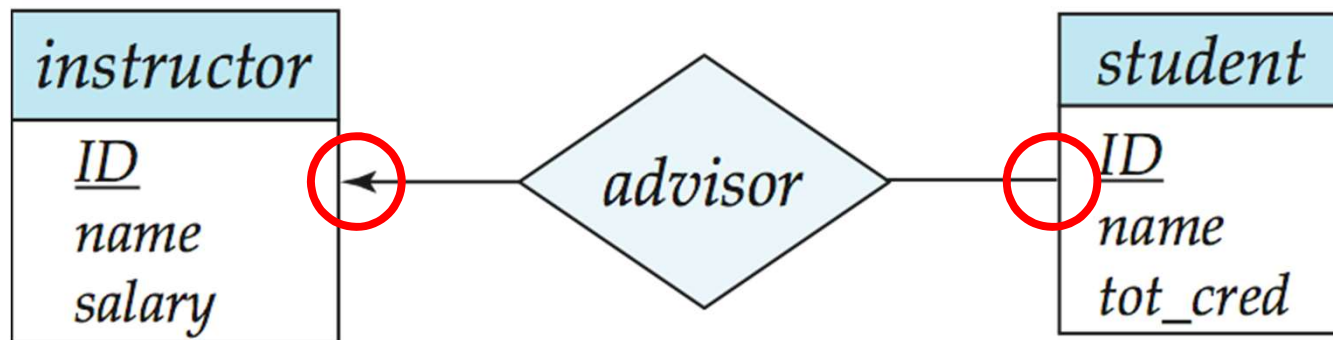
One-to-One Relationship

- ▶ **One-to-one** relationship between an *instructor* and a *student*
 - ▶ an instructor is associated with *at most one* student via *advisor*
 - ▶ and a student is associated with *at most one* instructor via *advisor*



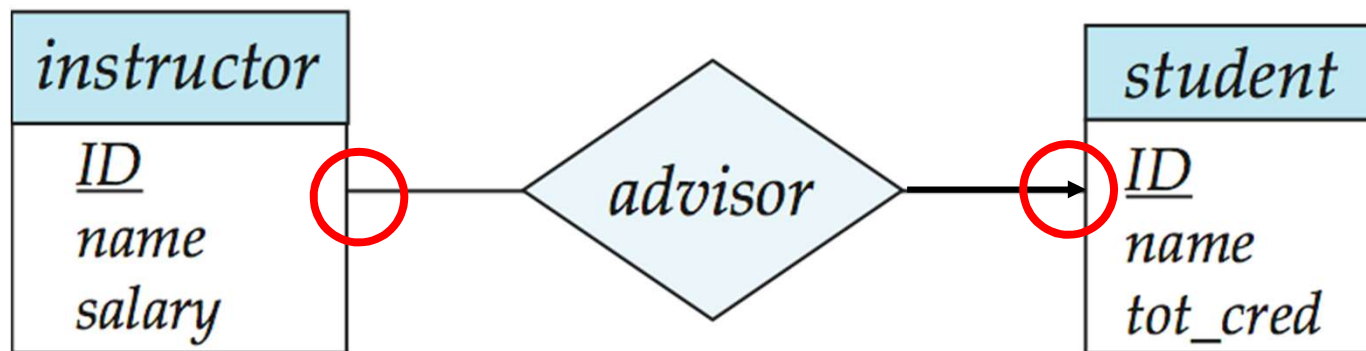
One-to-Many Relationship

- ▶ **One-to-many** relationship between an *instructor* and a *student*
 - ▶ an instructor is associated with *several (including 0)* students via *advisor*
 - ▶ a student is associated with *at most one* instructor via *advisor*



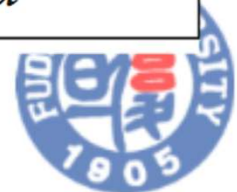
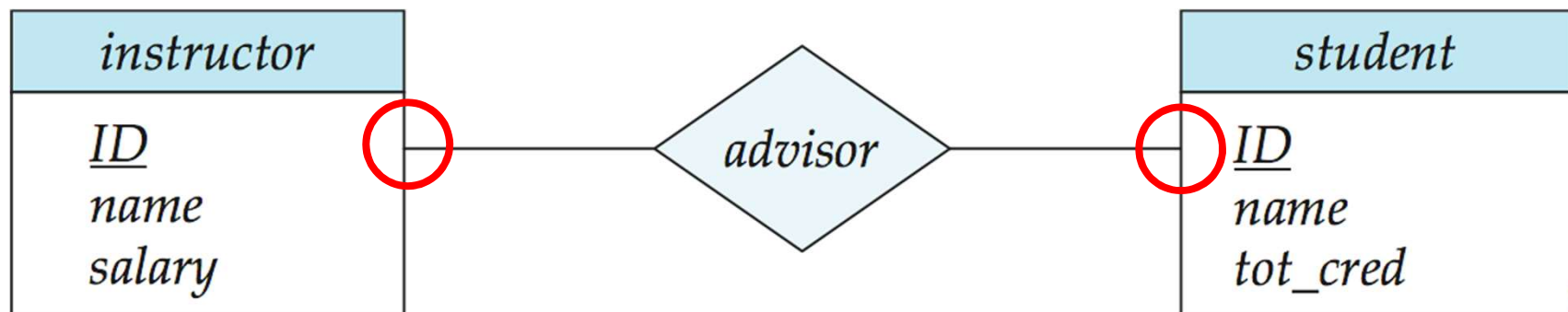
Many-to-One Relationships

- ▶ **Many-to-one** relationship between an *instructor* and a *student*
 - ▶ An instructor is associated with *at most one* student via *advisor*
 - ▶ A student is associated with *several (including 0)* instructors via *advisor*



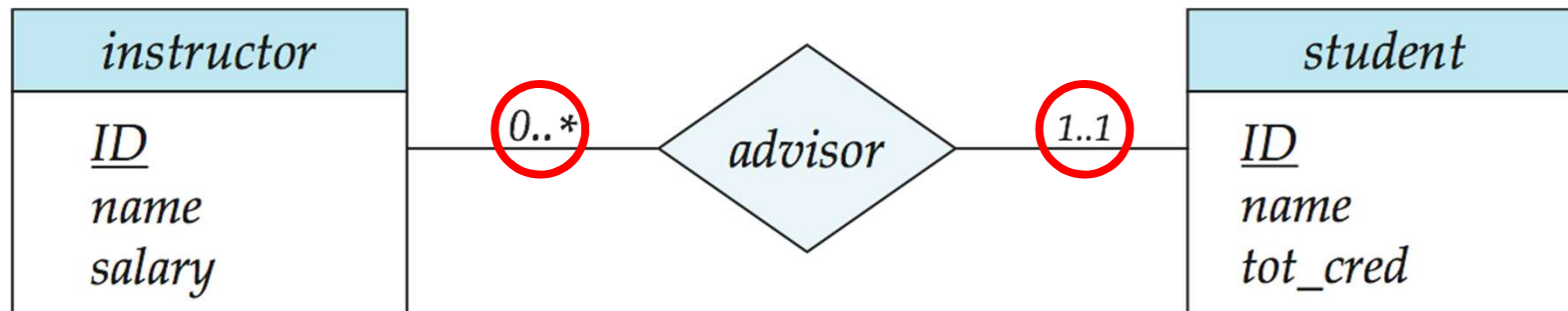
Many-to-Many Relationship

- ▶ **Many-to-Many** relationship between an *instructor* and a *student*
 - ▶ An instructor is associated with *several (possibly 0)* students via *advisor*
 - ▶ A student is associated with *several (possibly 0)* instructors via *advisor*

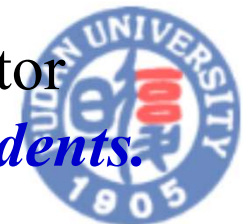


Alternative Notation for Cardinality Limits

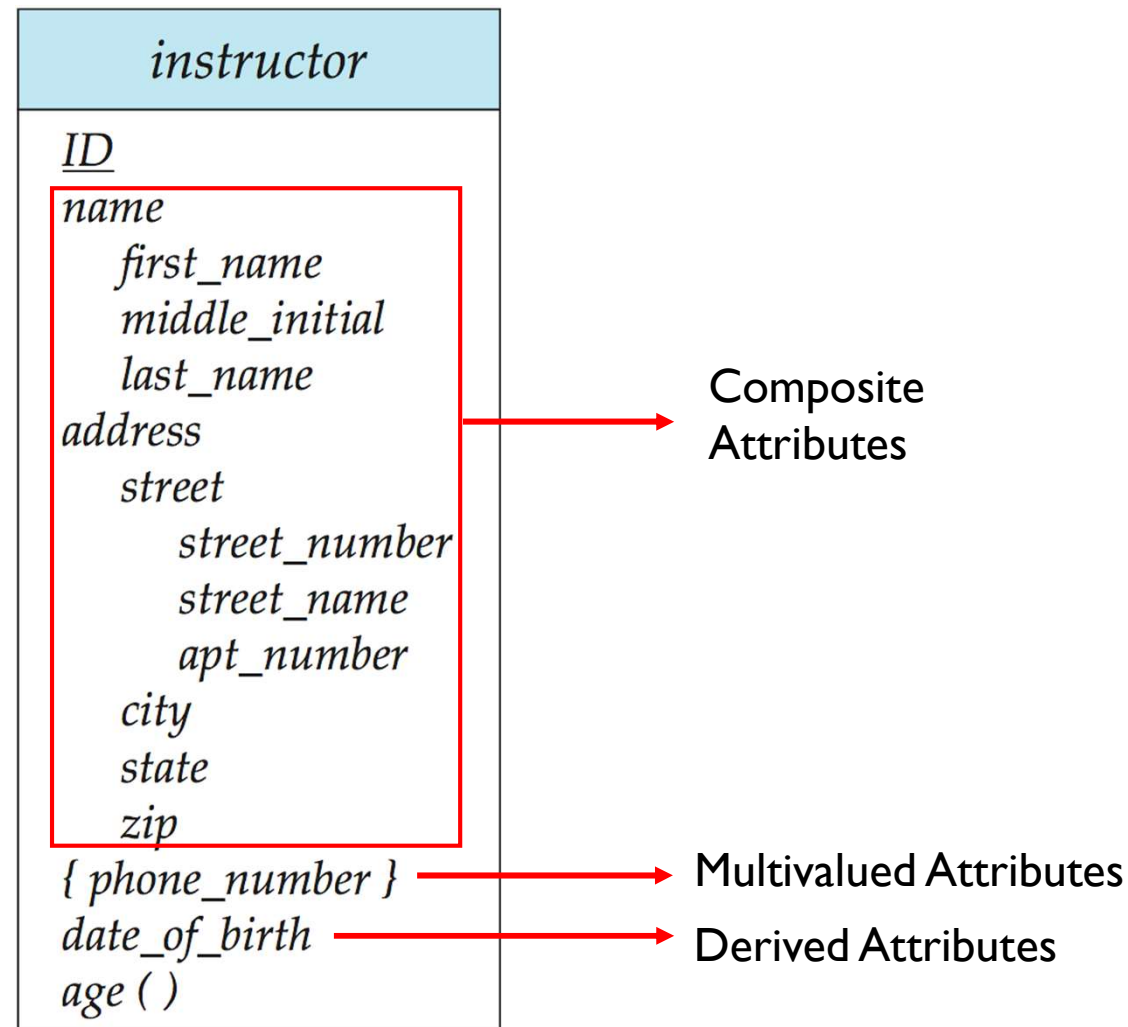
- ▶ A line may have an associated **minimum** and **maximum cardinality**, shown in the form *l..h*, where *l* is the minimum and *h* the maximum cardinality.



- ▶ The line between *advisor* and *student* has a cardinality constraint of **1.. 1**, meaning *the minimum and the maximum cardinality are both 1* (i.e. each student must have exactly one advisor).
- ▶ The limit **0 .. *** on the line between *advisor* and *instructor* indicates that *an instructor can have zero or more students*.

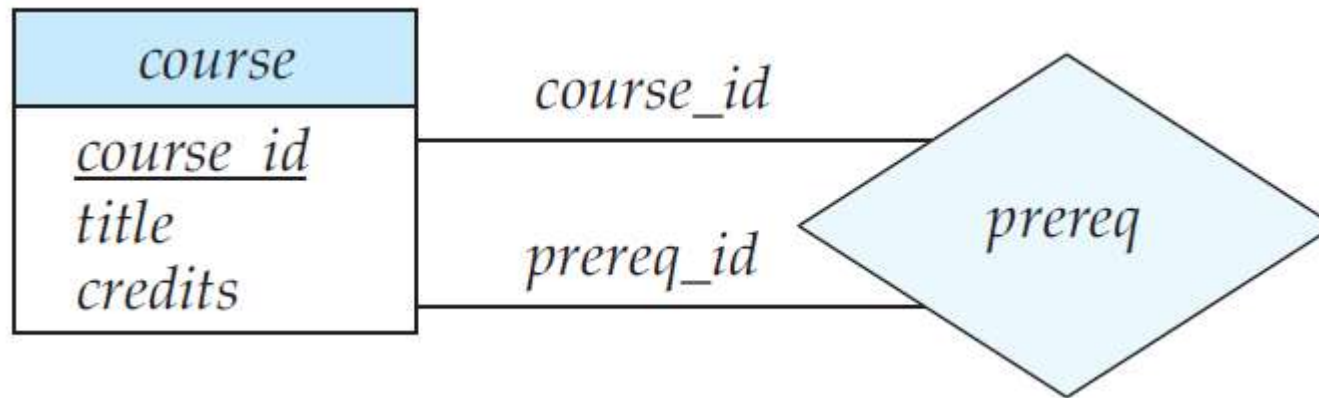


Composite, Multivalued, and Derived Attributes

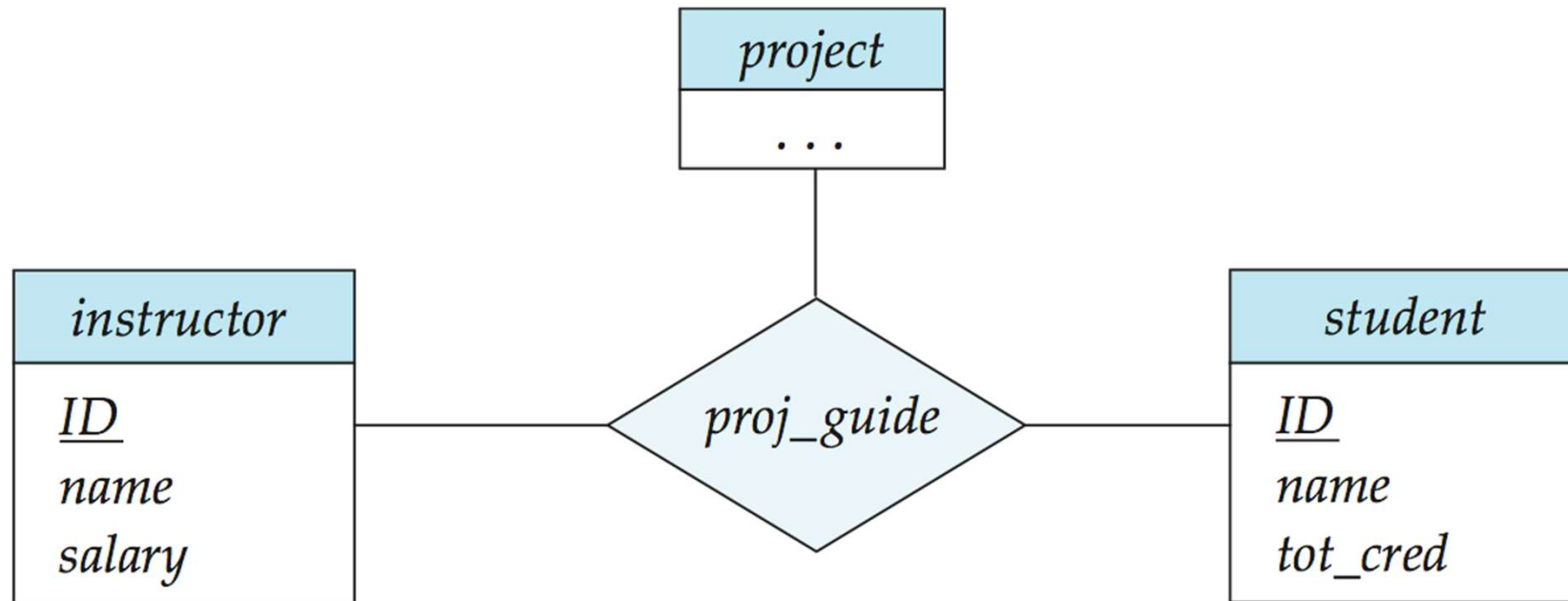


Roles

- ▶ The **role indicators** *course_id* and *prereq_id* between the *course* entity set and the *prereq* relationship set.

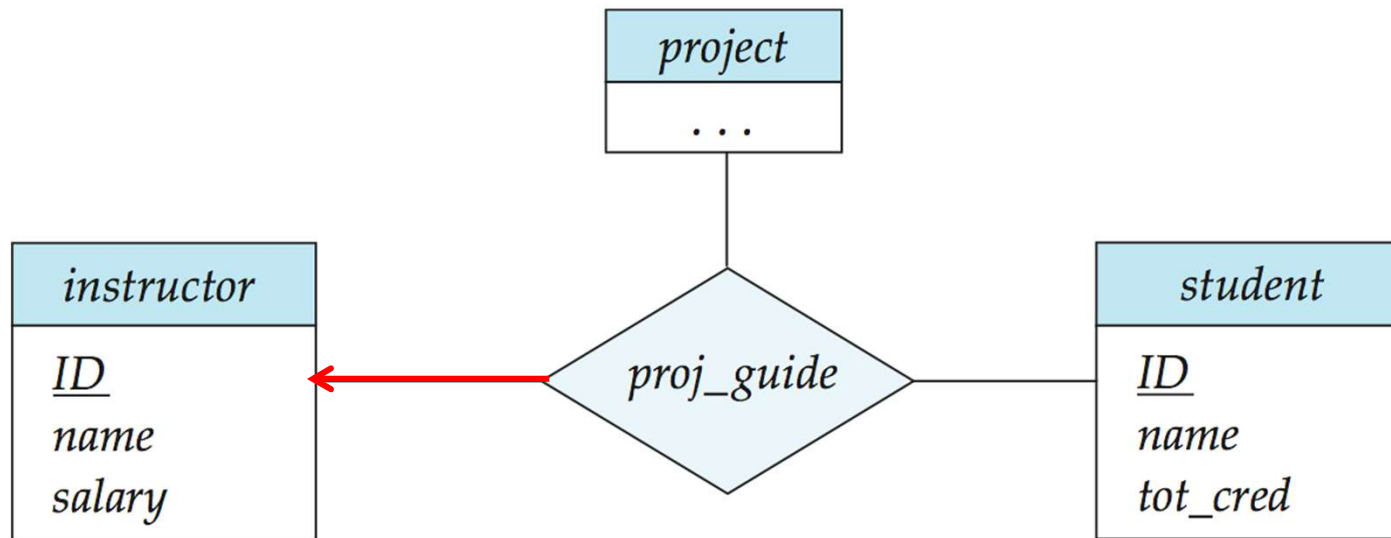


Nonbinary Relationship Sets



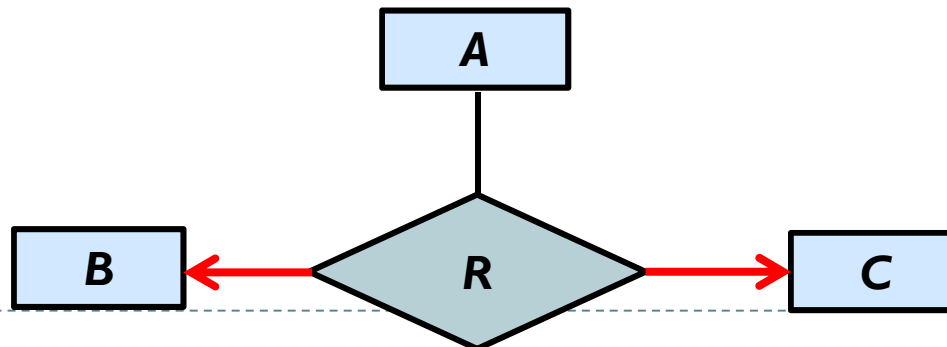
Cardinality Constraints on Ternary Relationship

- ▶ We allow **at most one arrow** out of a ternary (or greater degree) relationship to indicate a *cardinality constraint*
- ▶ E.g., an arrow from *proj_guide* to *instructor* indicates **each** student has **at most one** guide for **a** project



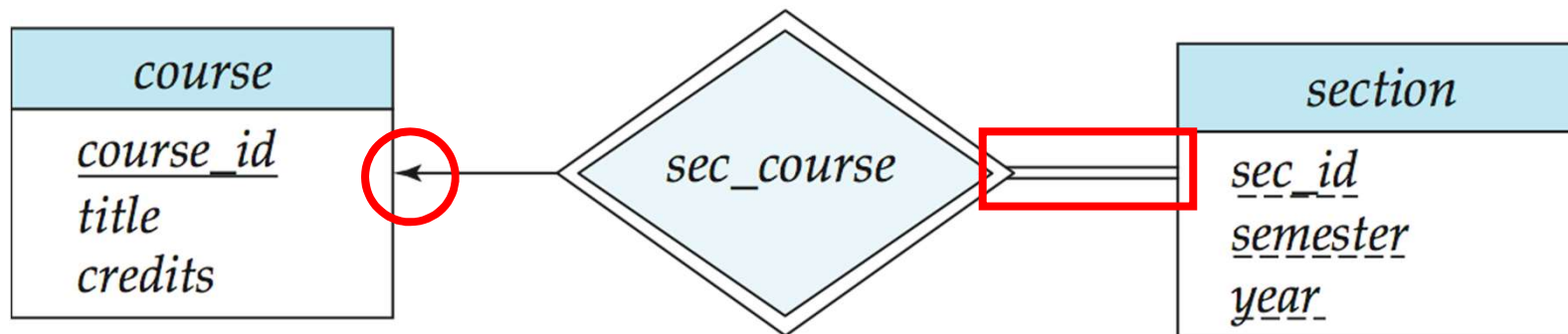
Cardinality Constraints on Ternary Relationship

- ▶ If there is **more than one arrow**, there are *two ways* of defining the meaning.
 - ▶ E.g., a ternary relationship R between A , B and C with arrows to B and C could mean
 1. *each A entity is associated with a unique entity from B and C* **or**
 2. *each pair of entities from (A, B) is associated with a unique C entity, and each pair (A, C) is associated with a unique B*
 - ▶ Each alternative has been used in different formalisms
 - ▶ To avoid confusion we **outlaw more than one arrow**



Participation of an Entity Set in a Relationship Set

- ▶ **Total participation** (indicated by **double line**): every entity in the entity set participates in *at least one* relationship in the relationship set
 - ▶ E.g.: participation of *section* in *sec_course* is total, i.e. every *section* must have an associated *course*

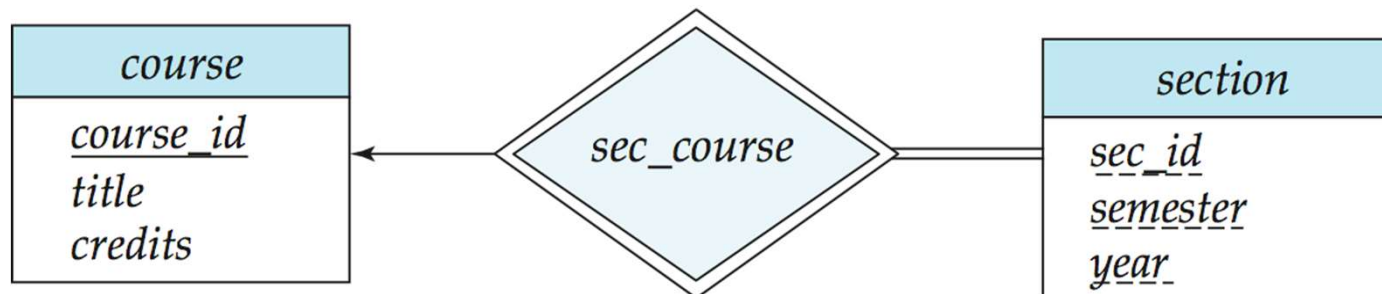


- ▶ **Partial participation**: some entities may not participate in any relationship in the relationship set
 - ▶ Example: participation of *instructor* in *advisor* is partial



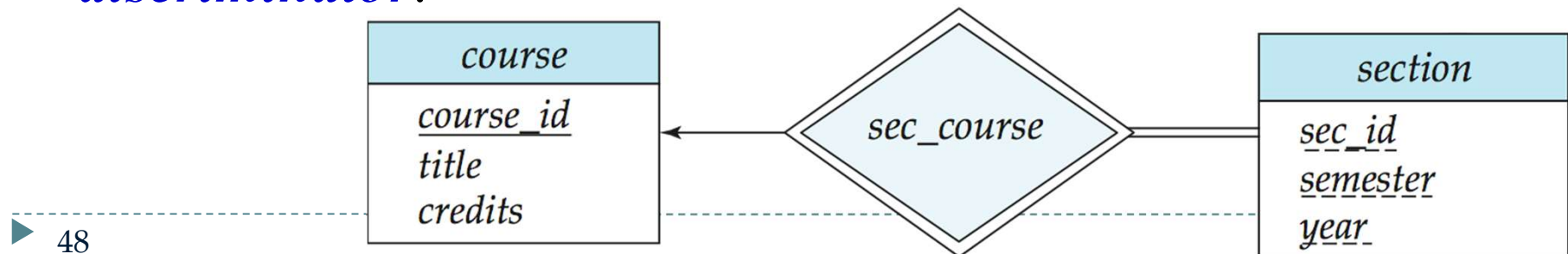
Weak Entity Sets

- ▶ An entity set that does **NOT** have a *primary key* is referred to as a **weak entity set**.
- ▶ The existence of a weak entity set depends on the existence of an **identifying or owner entity set**
- ▶ It must relate to the identifying entity set via a *total, one-to-many relationship set* from the identifying to the weak entity set
- ▶ **Identifying relationship** depicted using *a double diamond*



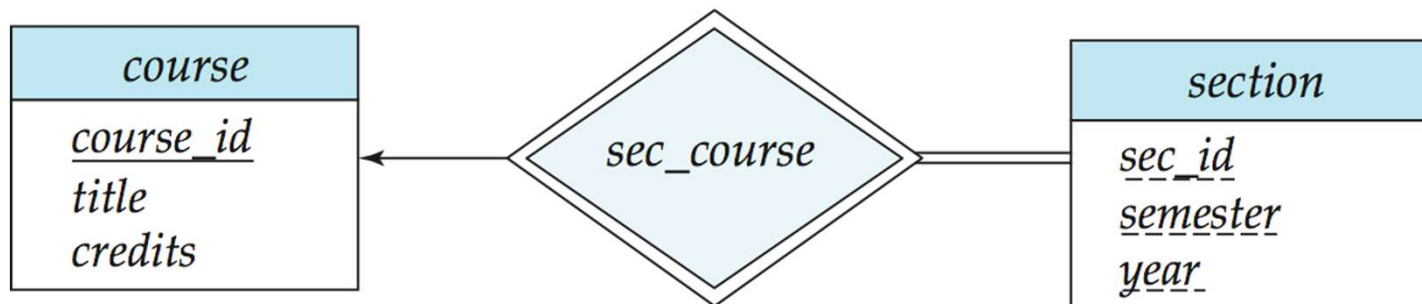
Weak Entity Sets

- ▶ The **discriminator** (分辨符) (or *partial key* 部分码) of a weak entity set is the set of attributes that distinguishes among all the entities of a weak entity set.
- ▶ Example: the discriminator of the weak entity set *section* :
 $\{sec\ id, year, semester\}$
- ▶ for **each course**, this set of attributes uniquely identifies one single *section* for that *course*.
- ▶ The **primary key of a weak entity set** is formed by the *primary key of the strong entity set* on which the weak entity set is existence dependent, **plus** the *weak entity set's discriminator*.



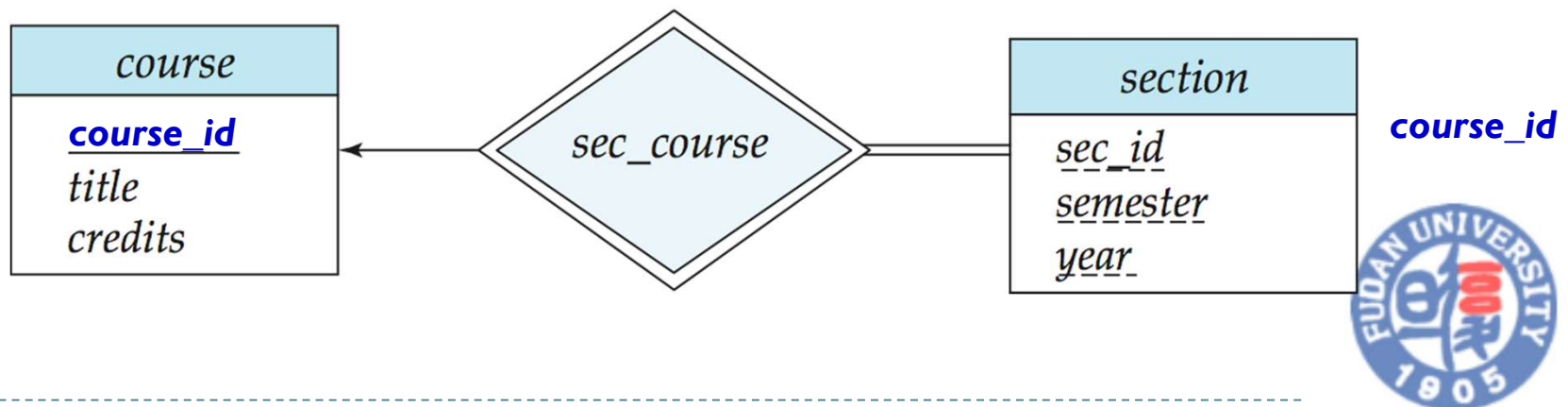
Weak Entity Sets (Cont.)

- ▶ We underline the discriminator of a weak entity set with *a dashed line*.
- ▶ We put the **identifying relationship** of a weak entity in **a double diamond**.
- ▶ Primary key for *section* – (*course_id*, *sec_id*, *semester*, *year*)



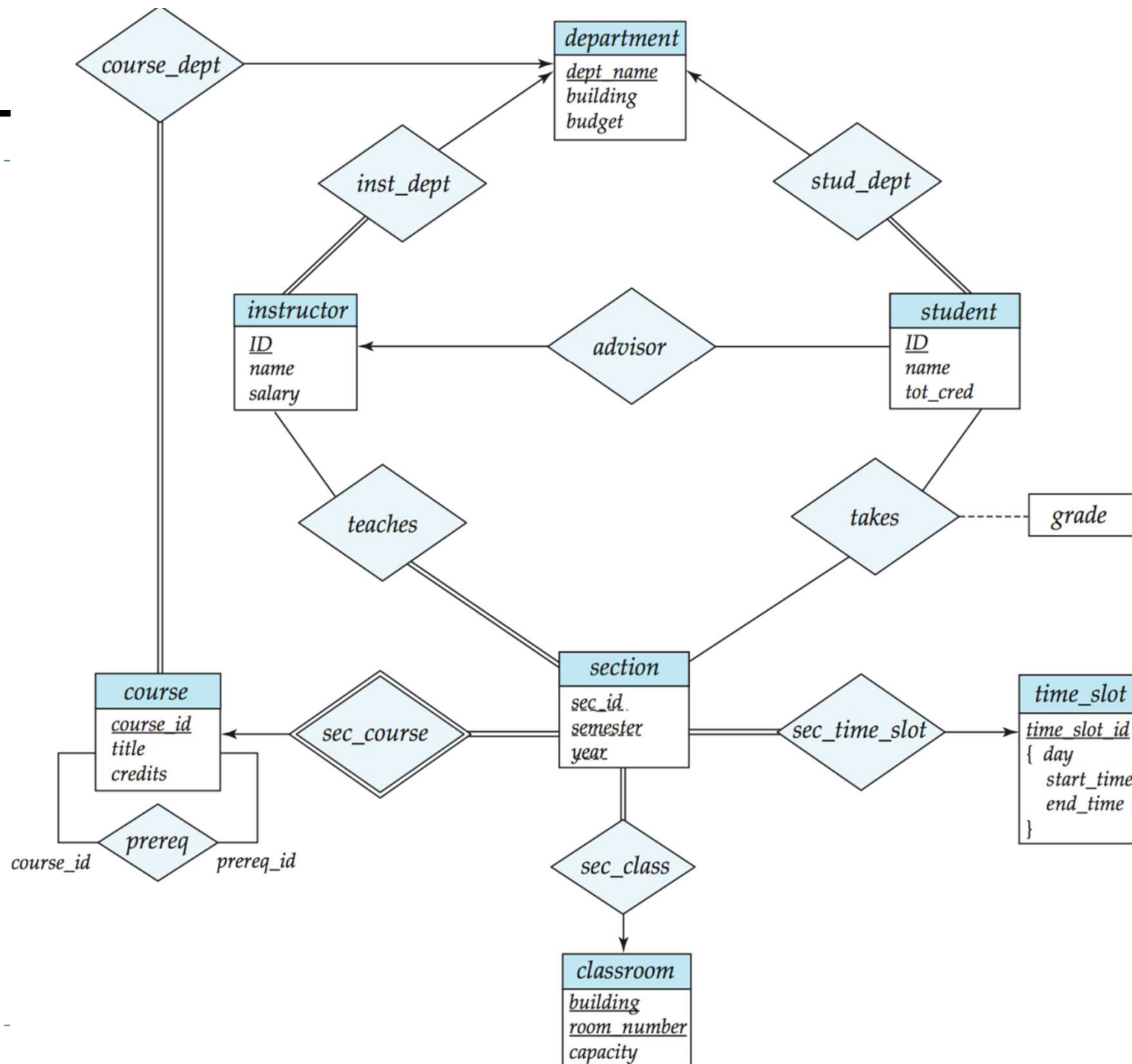
Weak Entity Sets (Cont.)

- ▶ **Note:** The **primary key** of the strong entity set is *NOT explicitly stored with the weak entity set*, since it is implicit in the identifying relationship.
- ▶ If *course_id* were explicitly stored, *section* could be made a strong entity, but then the relationship *sec_course* between *section* and *course* would be **duplicated** by an implicit relationship defined by the attribute *course_id* common to *course* and *section*



E-

se



Reduction to Relational Schemas

- ▶ **Entity sets** and **relationship sets** can be expressed uniformly as **relation schemas** that *represent the contents of the database*.
- ▶ A database which conforms to **an E-R diagram** can be represented by *a collection of schemas*.
- ▶ For **each entity set** and **relationship set** there is a *unique schema* that is assigned the name of the corresponding entity set or relationship set.
- ▶ Each schema has a number of **columns** (generally corresponding to *attributes*), which have *unique names*.



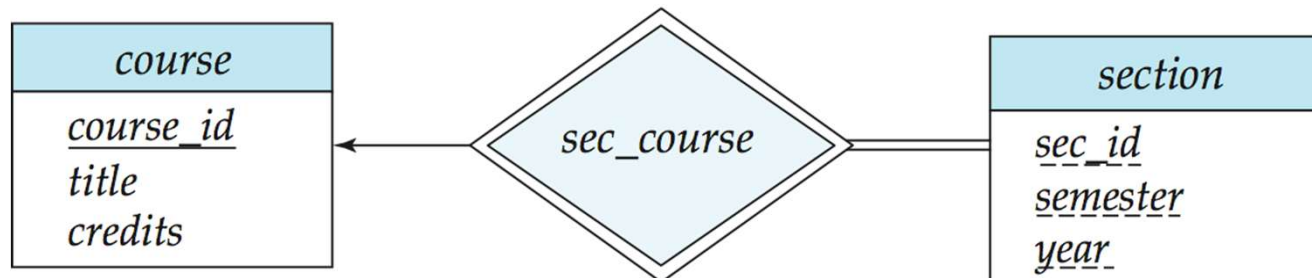
Representing Entity Sets with Simple Attributes

- ▶ A **strong entity set** reduces to a schema with the *same attributes*

course (*course_id*, *title*, *credits*)

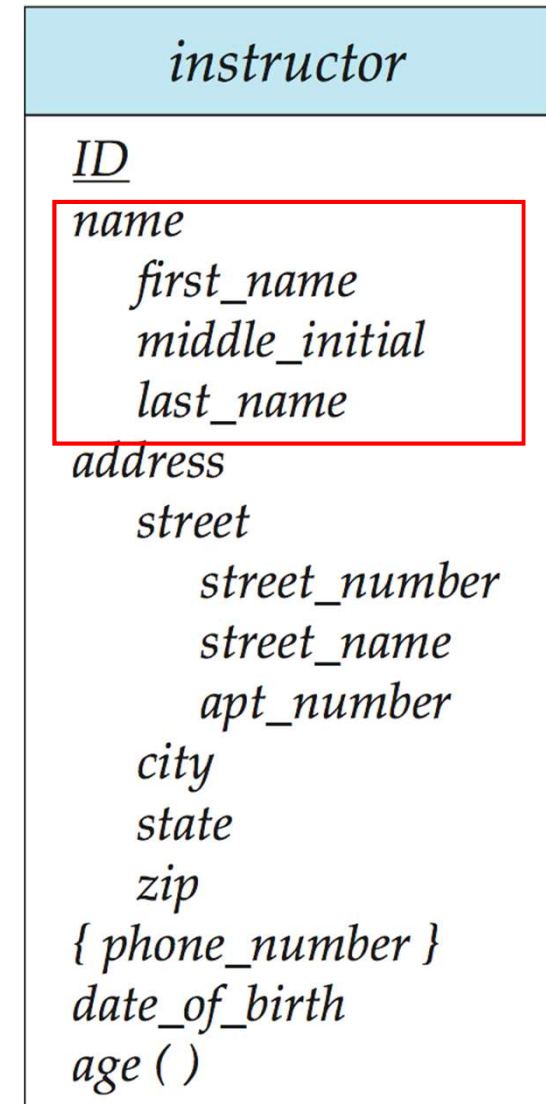
- ▶ A **weak entity set** becomes a table that includes a column for the primary key of the identifying strong entity set

section (*course_id*, *sec_id*, *sem*, *year*)



Composite and Multivalued Attributes

- ▶ **Composite attributes** are flattened out by creating a separate attribute for each component attribute
- ▶ Given entity set *instructor* with composite attribute *name* with component attributes *first_name*, *middle_initial* and *last_name*, the schema corresponding to the entity set has three attributes *name_first_name*, *name_middle_initial* and *name_last_name* (prefix omitted if there is no ambiguity)



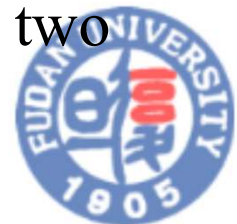
Composite and Multivalued Attributes

- ▶ Ignoring multivalued attributes, **extended** instructor schema is
 - ▶ ***instructor*** (*ID*,
first_name, *middle_initial*, *last_name*,
street_number, *street_name*, *apt_number*,
city, *state*, *zip_code*, *date_of_birth*)

<i>instructor</i>	
<u><i>ID</i></u>	
<i>name</i>	
<i>first_name</i>	
<i>middle_initial</i>	
<i>last_name</i>	
<i>address</i>	
<i>street</i>	
<i>street_number</i>	
<i>street_name</i>	
<i>apt_number</i>	
<i>city</i>	
<i>state</i>	
<i>zip</i>	
{ <i>phone_number</i> }	
<i>date_of_birth</i>	
<i>age</i> ()	

Composite and Multivalued Attributes

- ▶ A **multivalued attribute** M of an entity E is represented by a **separate schema** EM
 - ▶ Schema EM has attributes corresponding to the **primary key of E** and **an attribute corresponding to multivalued attribute M**
 - ▶ Multivalued attribute $phone_number$ of $instructor$ is represented by a schema:
$$inst_phone = (\underline{ID}, \underline{phone_number})$$
 - ▶ Each value of the multivalued attribute maps to a separate tuple of the relation on schema EM
 - ▶ For example, an $instructor$ entity with primary key 22222 and phone numbers 456-7890 and 123-4567 maps to two tuples:
$$(22222, 456-7890) \text{ and } (22222, 123-4567)$$



Representing Relationship Sets

- ▶ Let R be a **relationship set**.
 - ▶ let $a_1, a_2, \dots, a_m$ be the set of attributes formed by the union of the primary keys of each of the entity sets participating in R .
 - ▶ let the descriptive attributes (if any) of R be $b_1, b_2, \dots, b_n$.
 - ▶ a relation schema called R :

$$\{a_1, a_2, \dots, a_m\} \cup \{b_1, b_2, \dots, b_n\}$$

- ▶ The primary key is chosen as follows:
 - ▶ For a binary **many-to-many** relationship, the *union of the primary-key attributes* from the participating entity sets becomes the primary key.



Representing Relationship Sets (Cont.)

- ▶ For a binary **one-to-one** relationship set, the primary key of *either entity set* can be chosen as the primary key. The choice can be made arbitrarily.
- ▶ For a binary **many-to-one** or **one-to-many** relationship set, the *primary key of the entity set on the “many” side* of the relationship set serves as the primary key.
- ▶ For an **n -ary relationship set without any arrows on its edges**, the *union of the primary key-attributes* from the participating entity sets becomes the primary key.
- ▶ For an **n -ary relationship set with an arrow on one of its edges**, the *primary keys of the entity sets NOT on the “arrow” side* of the relationship set serve as the primary key for the schema (only one arrow allowed out of a relationship set).

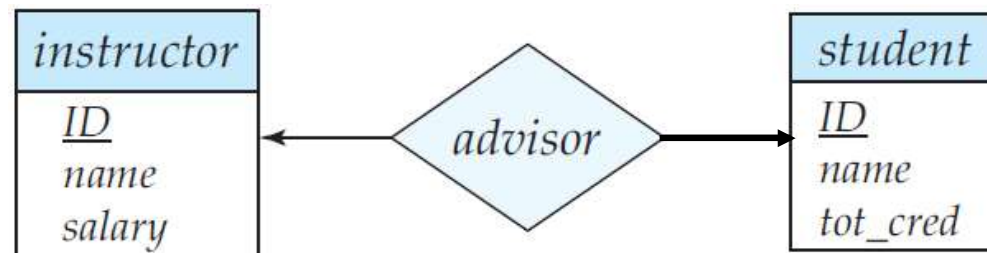


Representing Relationship Sets (Cont.)

- ▶ A **one-to-one relationship set** is represented as a schema with attributes for the primary keys of the two participating entity sets, and any descriptive attributes of the relationship set.
- ▶ Example: schema for relationship set *advisor*

$advisor = (\underline{s_id}, i_id)$ or $advisor = (s_id, \underline{i_id})$

Primary key



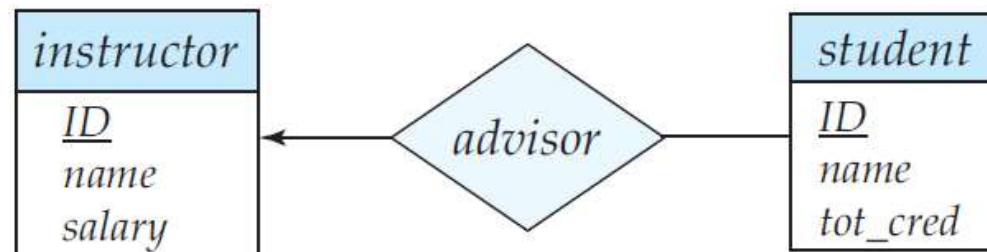
Representing Relationship Sets (Cont.)

- ▶ A **one-to-many** or **many-to-one relationship set** is represented as a schema with attributes for the primary keys of the two participating entity sets, and any descriptive attributes of the relationship set.
- ▶ Example: schema for relationship set *advisor*

$advisor = (\underline{s_id}, i_id)$



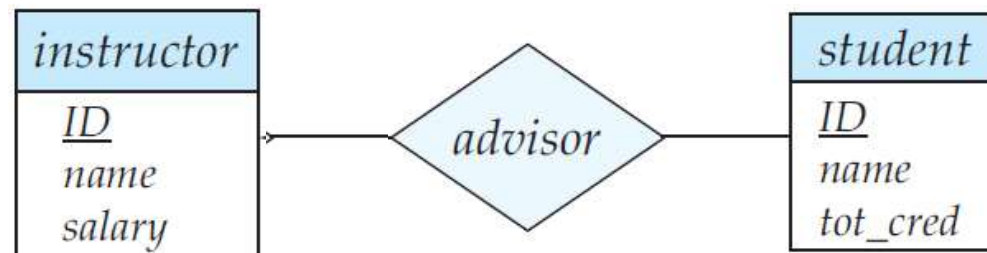
Primary key



Representing Relationship Sets (Cont.)

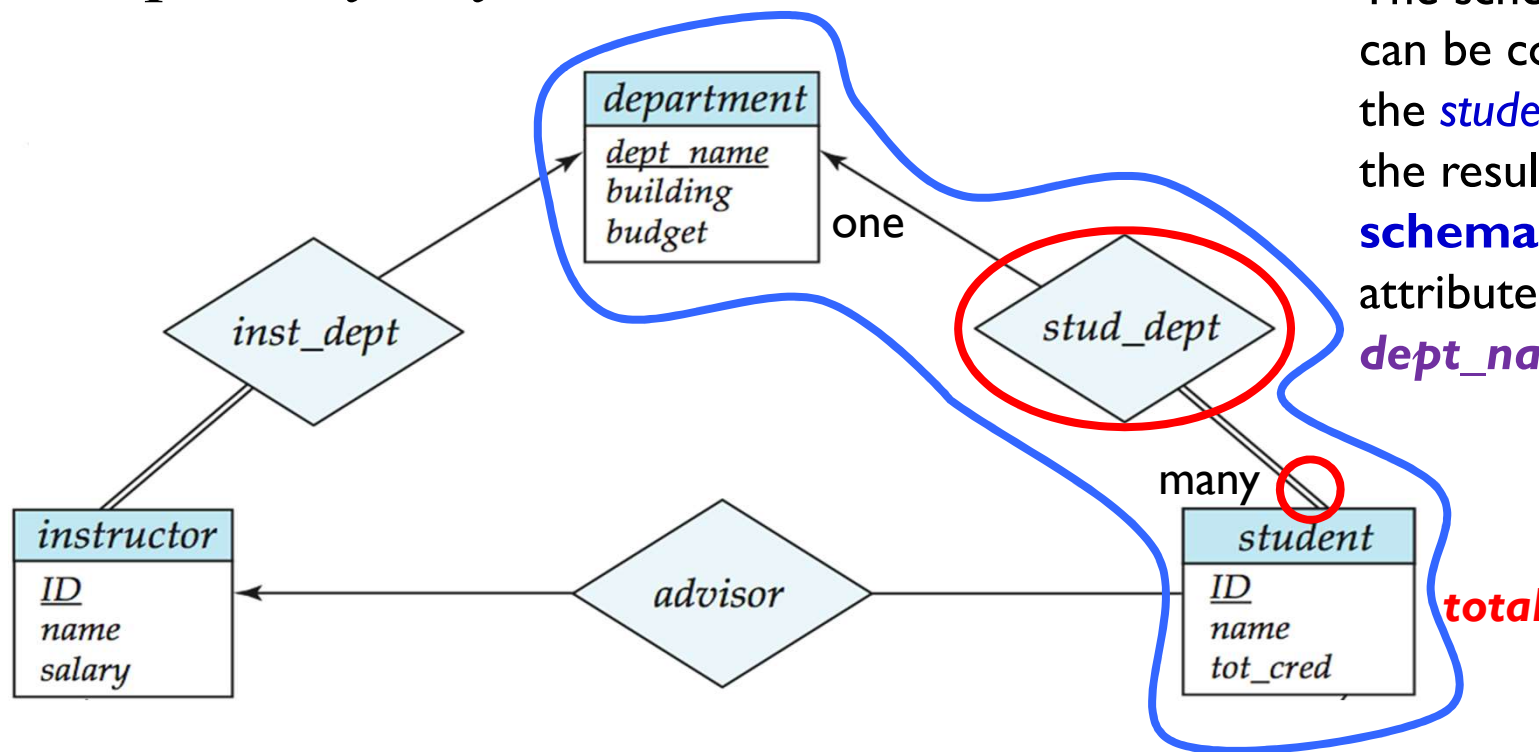
- ▶ A **many-to-many relationship set** is represented as a schema with attributes for the primary keys of the two participating entity sets, and any descriptive attributes of the relationship set.
- ▶ Example: schema for relationship set *advisor*

advisor = (*s_id*, *i_id*) → **Primary key**



Combination of Schemas

- **Many-to-one** and **one-to-many relationship sets** that are *total on the many-side* can be represented by adding an **extra attribute** to the “*many*” side, containing the primary key of the “one” side

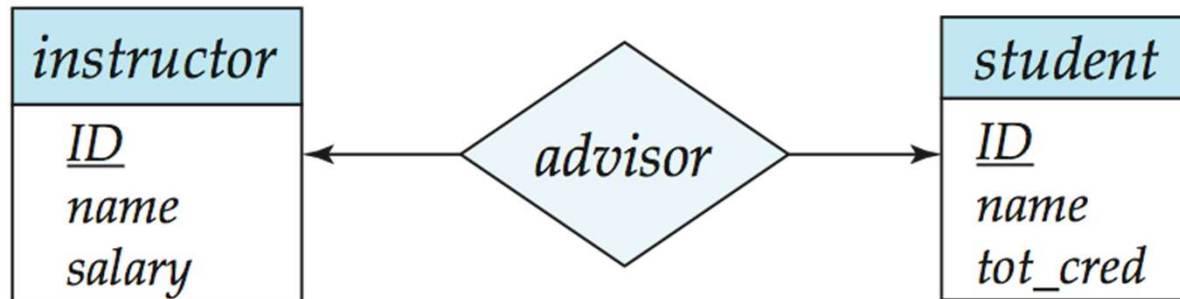


The schema **stud_dept** can be combined with the **student** schema, and the resulting **student schema** consists of the attributes {**ID**, **name**, **dept_name**, **tot_cred**}.



Combination of Schemas (Cont.)

- ▶ For **one-to-one relationship sets**, either side can be chosen to act as the “many” side
 - ▶ E.g. : Instead of creating a schema for relationship set *advisor*, extra attribute can be added to either of the tables corresponding to the two entity sets)

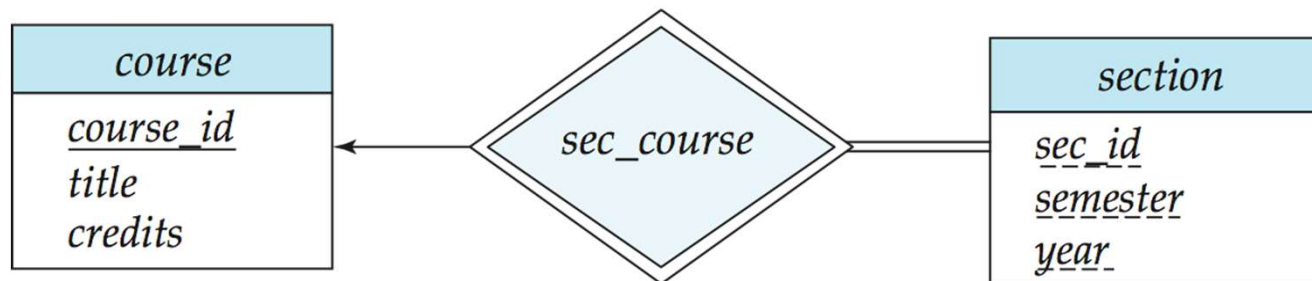


- ▶ If participation is **partial on the “many” side**, replacing a schema by an extra attribute in the schema corresponding to the “many” side could result in *null* values



Redundancy of Schemas

- ▶ The schema corresponding to *a relationship set linking a weak entity set to its identifying strong entity set* is **redundant**.
- ▶ This schema do **NOT need to be present** in a relational database design based upon an E-R diagram.
 - ▶ Example: The *section* schema already contains the attributes that would appear in the *sec_course* schema.
section : {*course_id*, *sec_id*, *semester*, *year*}
sec_course : {*course_id*, *sec_id*, *semester*, *year*}
- ▶ Thus, the *sec_course* schema is **redundant**.



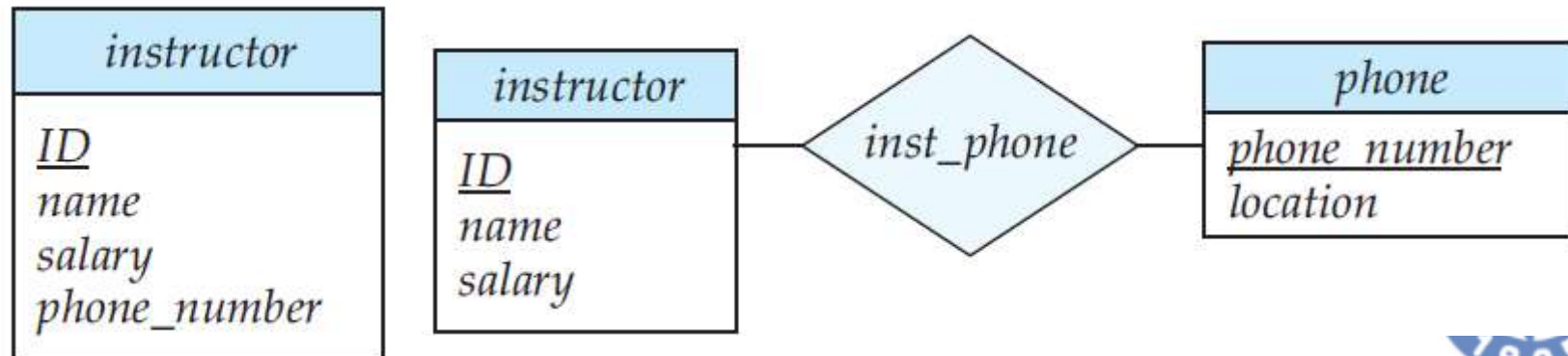
Design Issues

- ▶ The notions of *an entity set* and *a relationship set* are **NOT precise**, and it is possible to *define a set of entities and the relationships among them in a number of different ways*.



Use of Entity Sets vs. Attributes

- ▶ **Treating a phone as an attribute:** *phone_number* implies that *instructors* have precisely one *phone number* each.
- ▶ **Treating a phone as an entity:** *phone* permits *instructors* to have several *phone numbers* (including zero) associated with them.
- ▶ The **main difference** then is that treating a *phone* as an entity better models a situation where one may want to **keep extra information about a phone**, such as its location, or its type (mobile, IP phone, or plain old phone).



Use of Entity Sets vs. Attributes

- ▶ **What constitutes an attribute and an entity set?**

- ▶ The distinctions mainly depend on *the structure of the real-world enterprise* being modeled, and on *the semantics associated with the attribute* in question.

- ▶ A **common mistake** is to *use the primary key of an entity set as an attribute of another entity set, instead of using a relationship.*

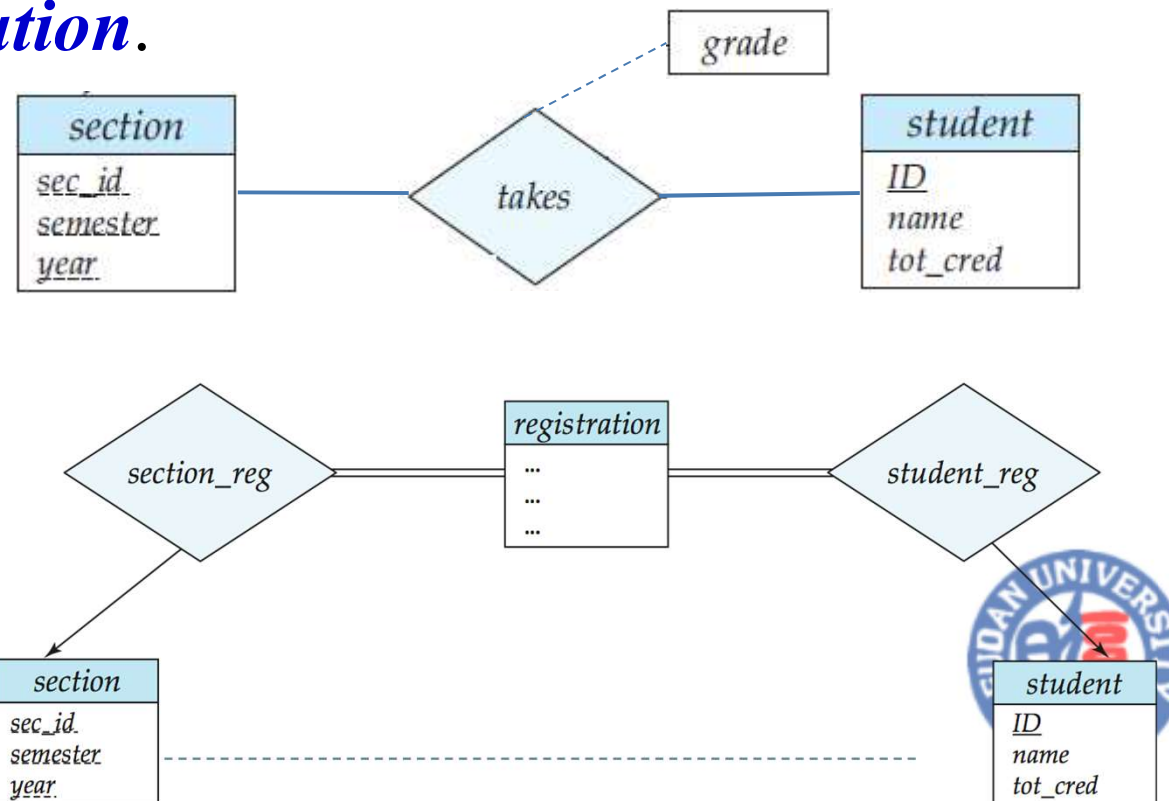
- ▶ For example, it is incorrect to model the *ID* of a *student* as an attribute of an *instructor* even if each *instructor* advises only one *student*.
- ▶ The relationship *advisor* is the correct way to represent the connection between *students* and *instructors*, since *it makes their connection explicit, rather than implicit via an attribute.*



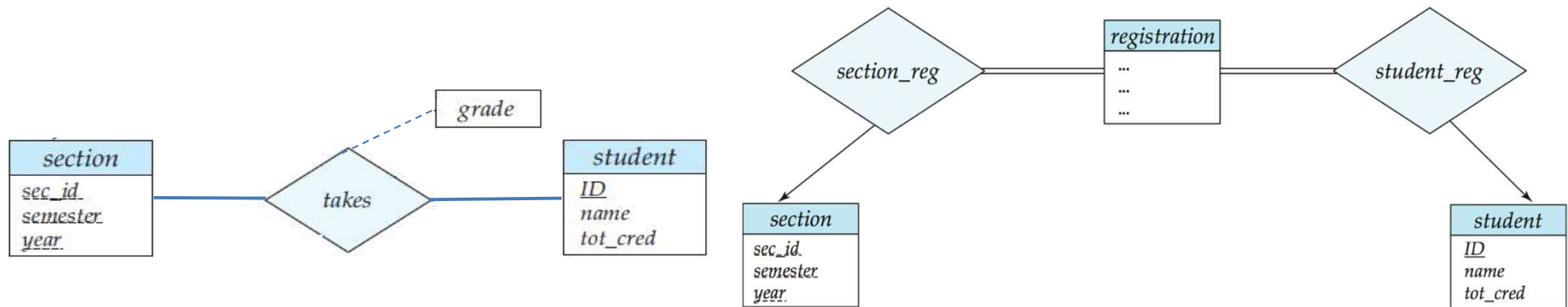
Use of Entity Sets vs. Relationship Sets

- ▶ We used to use the *takes* relationship set to model the situation where a *student* takes a *course*.
- ▶ Imagine that there is a **course-registration record** for each *course* that each *student* takes. Let us call that entity set *registration*.

Replacement of *takes*
by *registration* and
two *relationship sets*



Use of Entity Sets vs. Relationship Sets



- ▶ The use of *takes* is more compact and probably preferable. However, if the registrar's office associates **other information** with a *course-registration record*, it might be best to make it an entity.
- ▶ One **possible guideline** in determining whether to use an entity set or a relationship set is to **designate a relationship set to describe an ACTION that occurs between entities**.



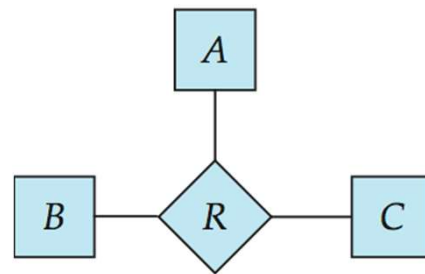
Binary Vs. Non-Binary Relationships

- ▶ Some relationships that appear to be **non-binary** may be better represented using **binary relationships**
 - ▶ E.g., A **ternary relationship** *parents*, relating a *child* to his/her *father* and *mother*, is best replaced by **two binary relationships**, *father* and *mother*
 - ▶ **Using two binary relationships allows partial information** (e.g., only *mother* being know)
 - ▶ But there are some relationships that are **naturally non-binary**
 - ▶ Example: *proj_guide*

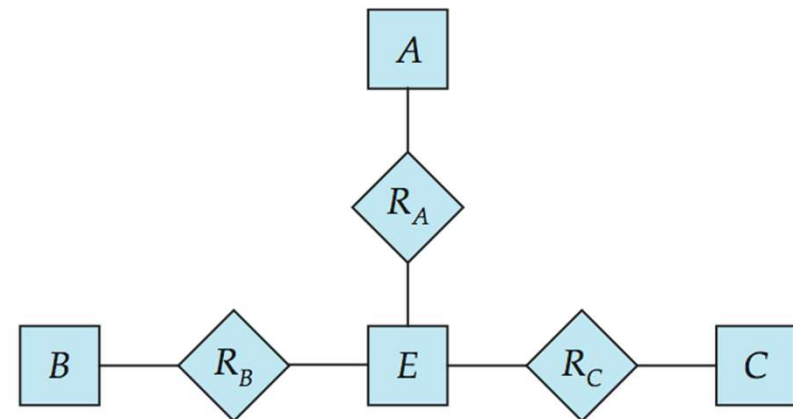


Converting Non-Binary Relationships to Binary Form

- ▶ In general, any non-binary relationship can be represented using binary relationships by **creating an artificial entity set**.
- ▶ Replace R between entity sets A , B and C by an **entity set E** , and **three relationship sets**:
 1. R_A , relating E and A
 2. R_B , relating E and B
 3. R_C , relating E and C



(a)

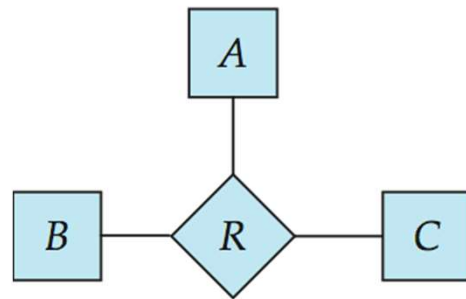


(b)

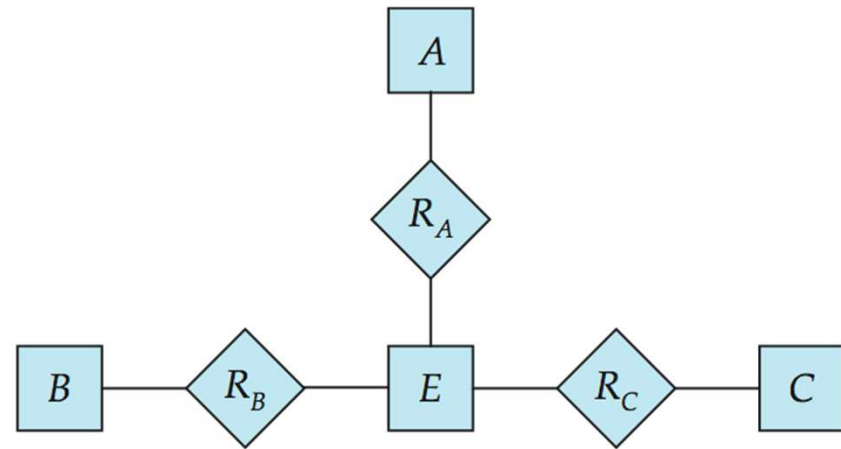


Converting Non-Binary Relationships to Binary Form

- ▶ Create a *special identifying attribute* for E
- ▶ Add any attributes of R to E
- ▶ For each relationship (a_i, b_i, c_i) in R , create
 1. a **new entity** e_i in the entity set E
 2. add (e_i, a_i) to R_A
 3. add (e_i, b_i) to R_B
 4. add (e_i, c_i) to R_C



(a)

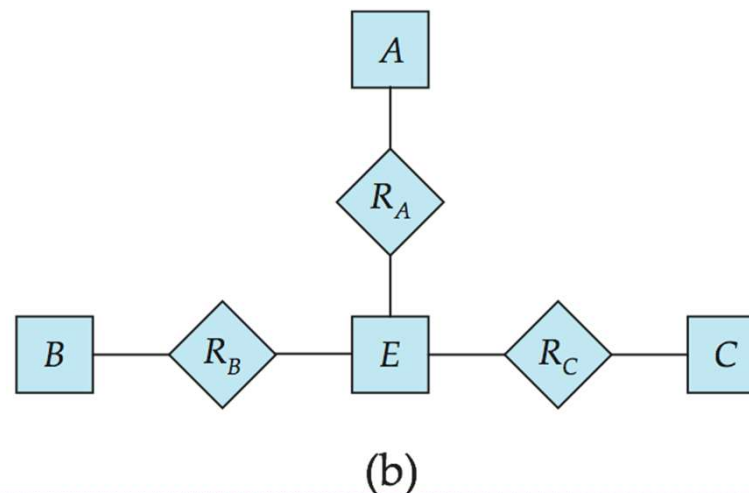
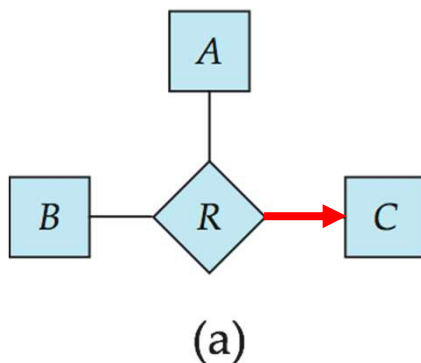


(b)



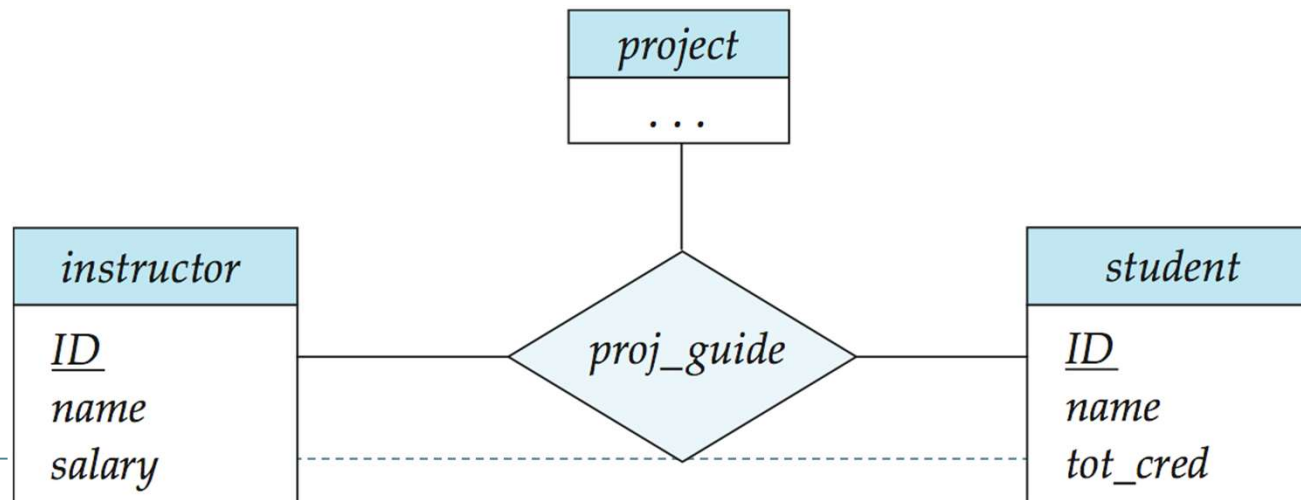
Converting Non-Binary Relationships

- ▶ We also need to translate constraints, but *translating all constraints may not be possible*
- ▶ For example, consider a constraint that says that R is many-to-one from A, B to C ; that is, each pair of entities from A and B is associated with at most one C entity. This constraint cannot be expressed by using cardinality constraints on the relationship sets R_A , R_B , and R_C .



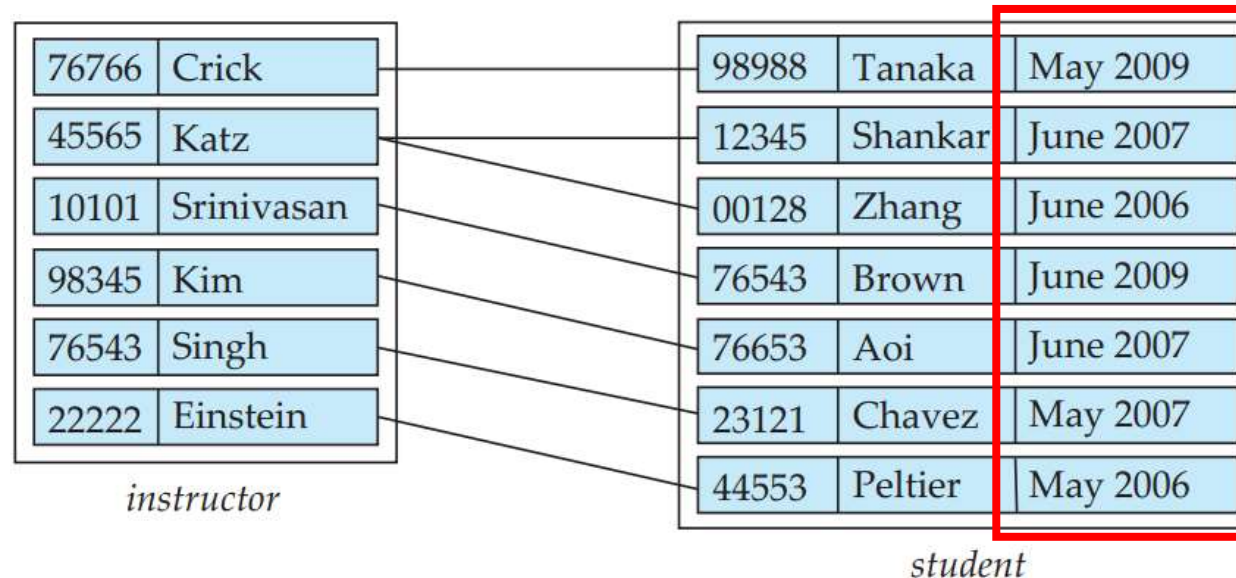
Converting Non-Binary Relationships

- ▶ Sometime, converting non-binary relationship into binary relationship *may be not natural*.
- ▶ Consider the relationship set *proj_guide*. If we directly split *proj_guide* into binary relationships. We would be able to record that instructor *Katz* works on projects *A* and *B* with students *Shankar* and *Zhang*; however, we would not be able to record that *Katz* works on project *A* with student *Shankar* and works on project *B* with student *Zhang*.



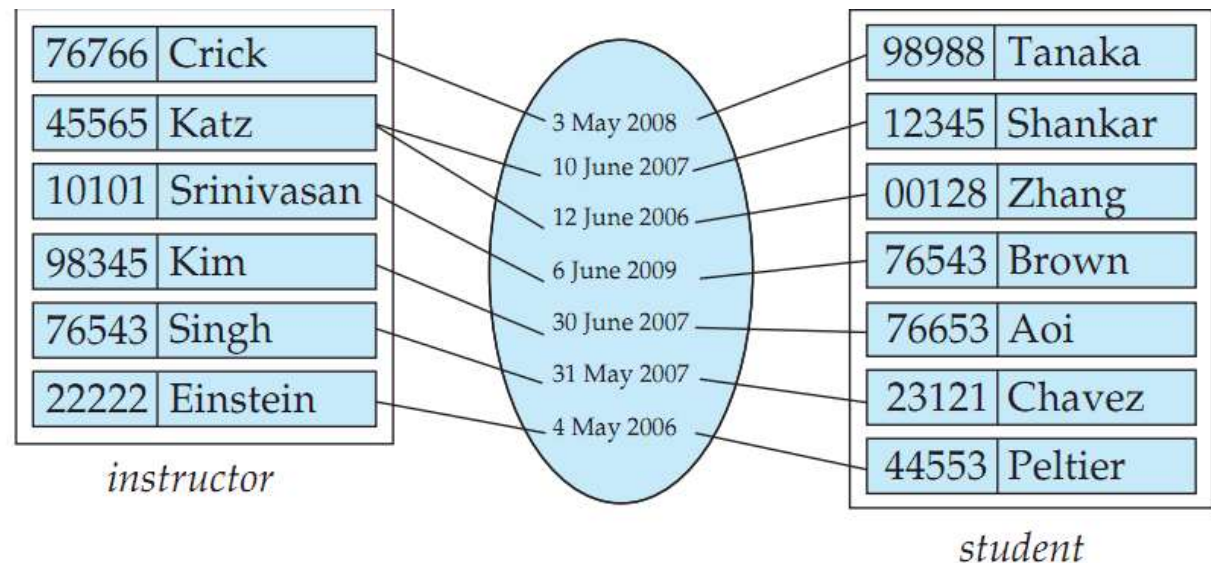
Placement of Relationship Attributes

- ▶ The **cardinality ratio of a relationship** can affect the *placement of relationship attributes*.
- ▶ Thus, **attributes** of *one-to-one* or *one-to-many* relationship sets can be associated with *one of the participating entity sets*, rather than with the *relationship set*.
 - ▶ e.g., if *advisor* is a **one-to-many** relationship set
 - ▶ attribute *date* as attribute of *advisor* or as attribute of *student*



Placement of Relationship Attributes

- ▶ When an **attribute** is determined by the *combination of participating entity sets*, rather than by *either entity separately*, that attribute must be associated with the **many-to-many relationship set**.
- ▶ E.g.: if *advisor* is a **many-to-many** relationship set
- ▶ If *date* were an attribute of *student*, for instance, we could not determine which *instructor* became the *advisor* on that particular *date*.

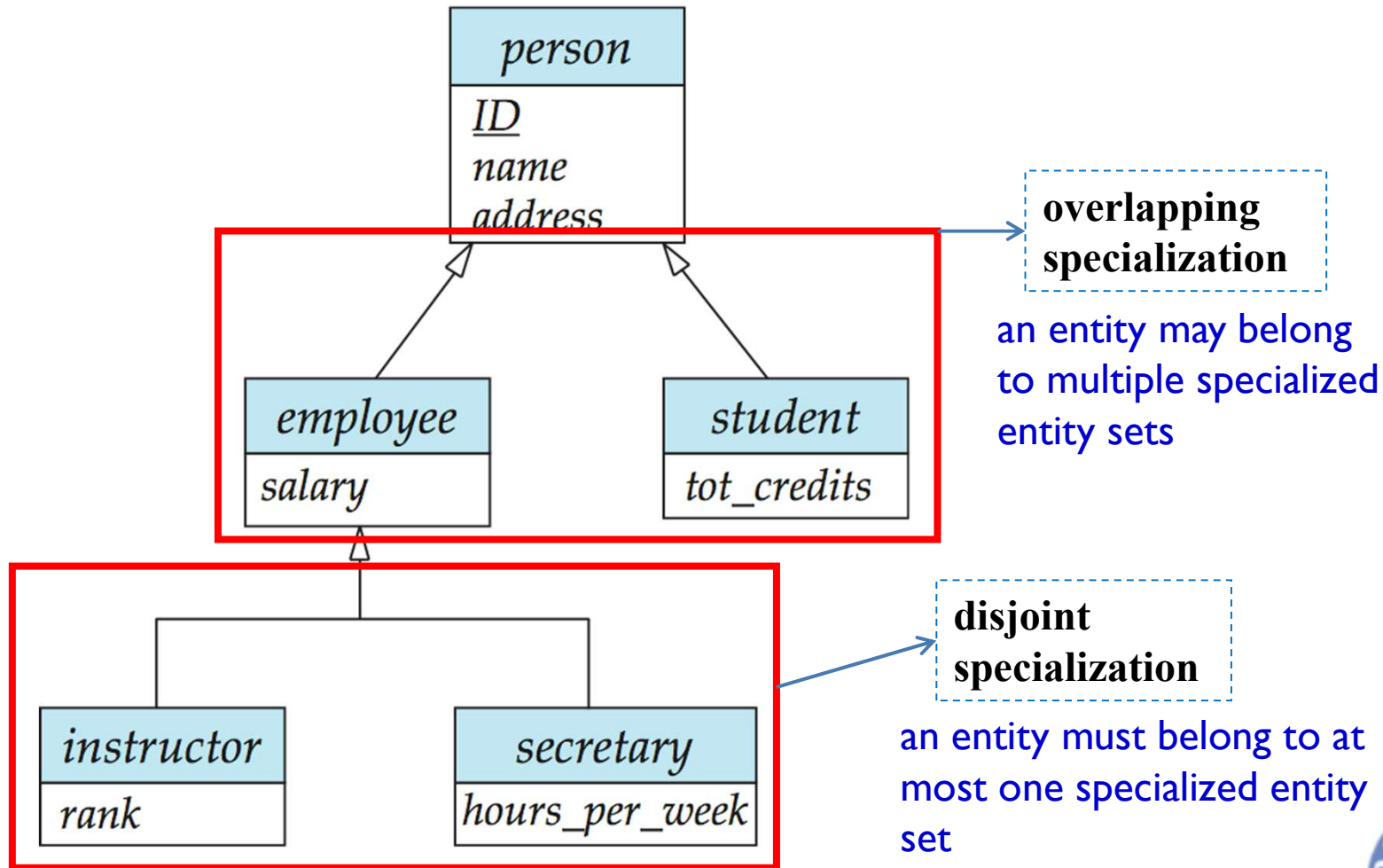


Extended E-R Features: Specialization

- ▶ **Specialization - top-down design process**: we designate *subgroupings* within an entity set that are distinctive from other entities in the set.
- ▶ These subgroupings become *lower-level entity sets* that have attributes or participate in relationships that do not apply to the higher-level entity set.
- ▶ In terms of an E-R diagram, specialization is depicted by a *hollow arrow-head pointing* from the specialized entity to the other entity. We refer to this relationship as the **ISA relationship**. (E.g., *instructor* “is a” *person*).
- ▶ **Attribute inheritance** – a lower-level entity set inherits all the attributes and relationship participation of the higher-level entity set to which it is linked.



Specialization Example



Extended E-R Features: Generalization

- ▶ **Generalization** - A bottom-up design process :
combine a number of entity sets that *share the same features* into a higher-level entity set.
- ▶ E.g.
 - ▶ *instructor* entity set with attributes *instructor_id* , *instructor name*, *instructor salary*, and *rank*.
 - ▶ *secretary* entity set with attributes *secretary_id* , *secretary name*, *secretary salary*, and *hours_per_week*.
 - ▶ generalize the higher-level entity set: *employee*, with the attributes *ID*, *name*, *salary*.



Specialization and Generalization

- ▶ The **ISA relationship** also referred to as *superclass - subclass relationship*
- ▶ **Specialization** and **generalization** are simple inversions of each other; they are represented in an *E-R* diagram in the same way.
- ▶ The terms specialization and generalization are used *interchangeably*.



Constraints on Generalizations

- ▶ One type of **constraint** involves determining *which entities can be members of a given lower-level entity set*.
- ▶ **Condition-defined**
 - ▶ In *condition-defined lower-level entity sets*, membership is evaluated on the basis of whether or not an entity satisfies an explicit condition or predicate.
 - ▶ Example: all students with attribute *student_type* being “graduate” are members of *graduate_student* entity set
- ▶ **User-defined**
 - ▶ *User-defined lower-level entity sets* are not constrained by a membership condition; rather, the database user assigns entities to a given entity set.



Constraints on Generalizations

- ▶ The second type of **constraint** on *whether or not entities may belong to more than one lower-level entity set within a single generalization*.
- ▶ **Disjoint:** *an entity can belong to only one lower-level entity set*
 - ▶ E.g. *student* entity can satisfy only one condition for the *student_type* attribute; an entity can be either a *graduate student* or an *undergraduate student*, but cannot be both
- ▶ **Overlapping:** *an entity can belong to more than one lower-level entity set*
 - ▶ E.g. assuming certain employees participate in more than one work team; a given *employee* may appear in more than one of the team entity sets that are lower-level entity sets of *employee*



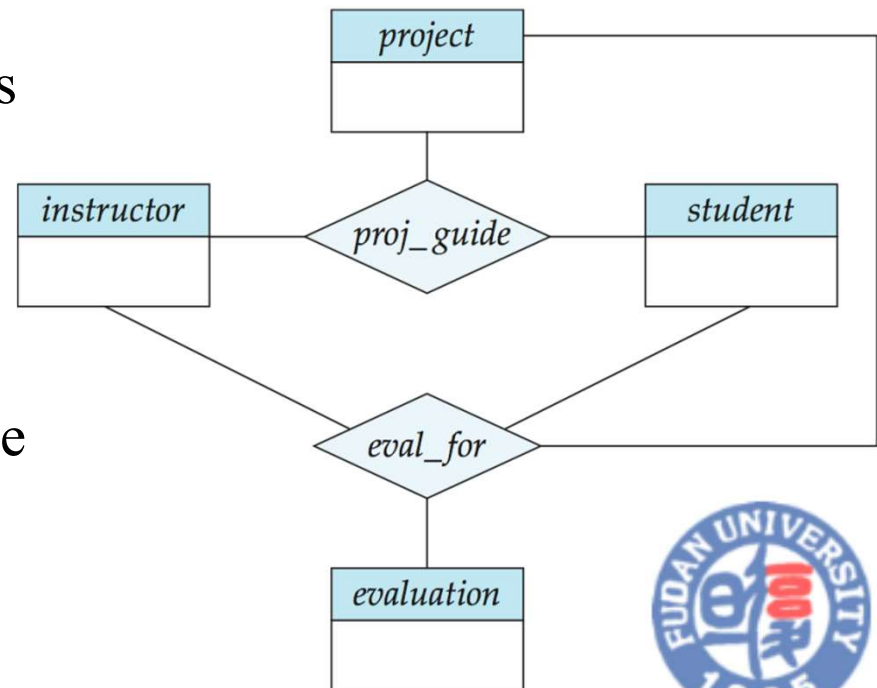
Constraints on Generalizations

- ▶ **Completeness constraint** - generalization or specialization specifies *whether or not an entity in the higher-level entity set must belong to at least one of the lower-level entity sets within a generalization.*
- ▶ **Total:** *must belong to one of the lower-level entity sets*
 - ▶ E.g. The generalization of *graduate student* and *undergraduate student* into *student* is a total, disjoint generalization.
- ▶ **Partial:** *need not belong to one of the lower-level entity sets*
 - ▶ E.g. Employees are assigned to a team only after 3 months on the job, some *employee* entities may not be members of any of the lower-level team entity sets.
- ▶ We can specify **total generalization** in an *E-R* diagram by *adding the keyword “total ” in the diagram.*



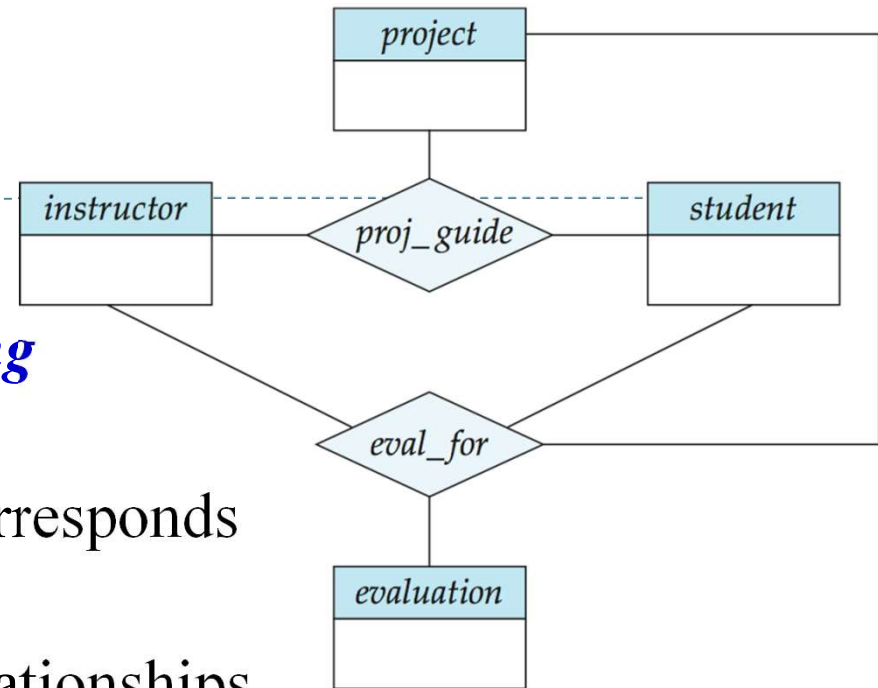
Aggregation

- ▶ One **limitation** of the *E-R* model is that it *cannot express relationships among relationships*. (e.g. ternary relationship *proj_guide*)
- ▶ Suppose we want to record evaluations of a *student* by a *guide* on a *project*
 - ▶ We model the evaluation report as an entity *evaluation*. One alternative for recording the *(student, project, instructor)* combination to which an evaluation corresponds is to create a quaternary (4-way) relationship set *eval_for* between *instructor*, *student*, *project*, and *evaluation*.



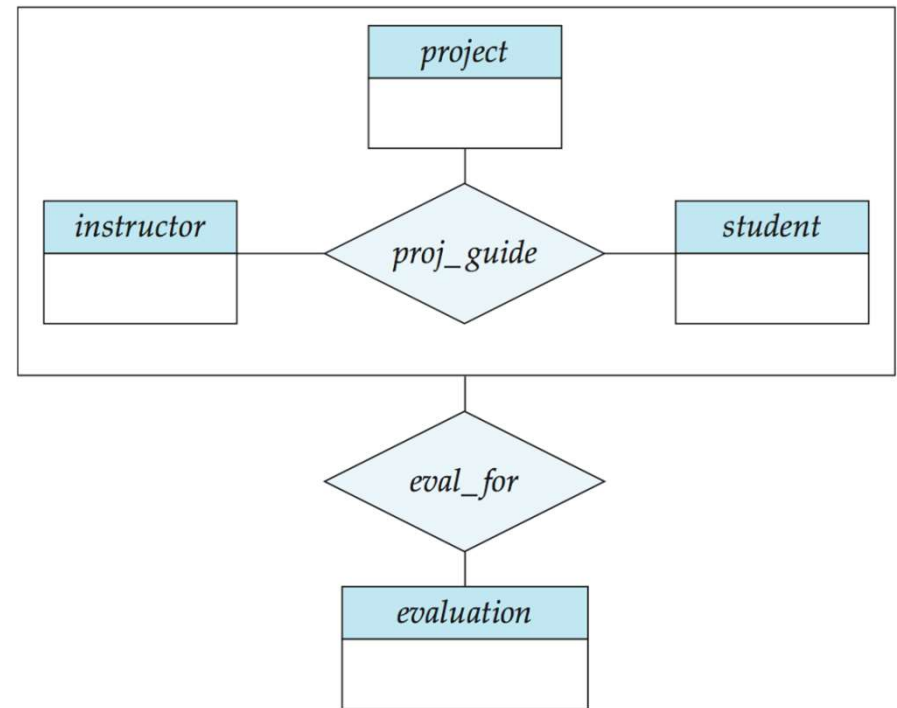
Aggregation (Cont.)

- ▶ Relationship sets *eval_for* and *proj_guide* represent *overlapping information*
- ▶ Every *eval_for* relationship corresponds to a *proj_guide* relationship
- ▶ However, some *proj_guide* relationships may not correspond to any *eval_for* relationships. So we can't discard the *proj_guide* relationship
- ▶ The best way to model this situation is to use **aggregation**.
- ▶ Aggregation is *an abstraction through which relationships are treated as higher-level entities*.



Aggregation (Cont.)

- ▶ We regard the relationship set *proj_guide* (relating the entity sets *instructor*, *student*, and *project*) as a higher-level entity set called *proj_guide*.
- ▶ We can then create a binary relationship *eval_for* between *proj_guide* and *evaluation* to represent which (*student*, *project*, *instructor*) combination an *evaluation* is for.



Reduction to Relation Schemas

Representation of Generalization

► Method 1

- Form *a schema* for the *higher-level entity*
- Form *a schema* for *each lower-level entity set*, include *primary key of higher-level entity set* and *local attributes*

person (ID, name, street, city)
employee (ID, salary)
student (ID, tot_cred)

- **Drawback:** getting information about an *employee* requires *accessing two relations* (the one corresponding to the low-level schema and the one corresponding to the high-level schema)



Reduction to Relation Schemas

► Method 2:

- Form *a schema* for *each entity set* with all local and inherited attributes

schema

attributes

person

ID, name, street, city

student

ID, name, street, city, tot_cred

employee

ID, name, street, city, salary

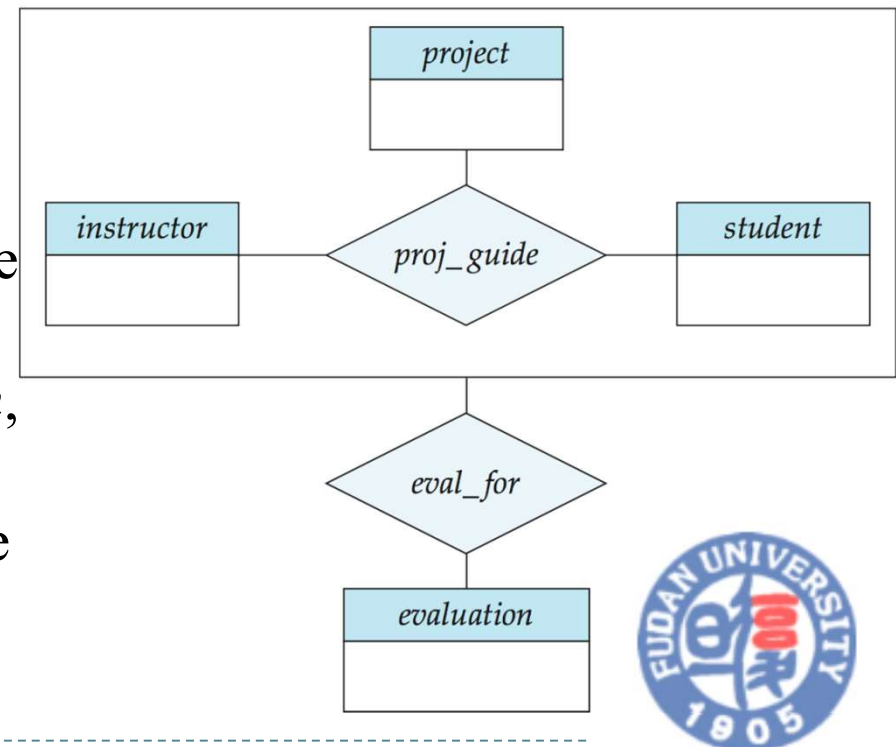
- If *specialization is total*, the schema for the generalized entity set (*person*) not required to store information
 - can be defined as *a “view” relation* containing union of specialization relations
 - but explicit schema may still be needed for *foreign key constraints*.
- **Drawback:** *name, street* and *city* may be stored redundantly for people who are both *students* and *employees*



Reduction to Relation Schemas

Representation of Aggregation

- ▶ To represent **aggregation**, create a schema containing
 - ▶ *primary key* of the aggregated relationship,
 - ▶ *primary key* of the associated entity set
 - ▶ *any descriptive attributes*
- ▶ The schema for the relationship set *eval_for* includes an attribute for each attribute in the primary keys of the entity set *evaluation*, and the relationship set *proj_guide*, and any descriptive attributes.

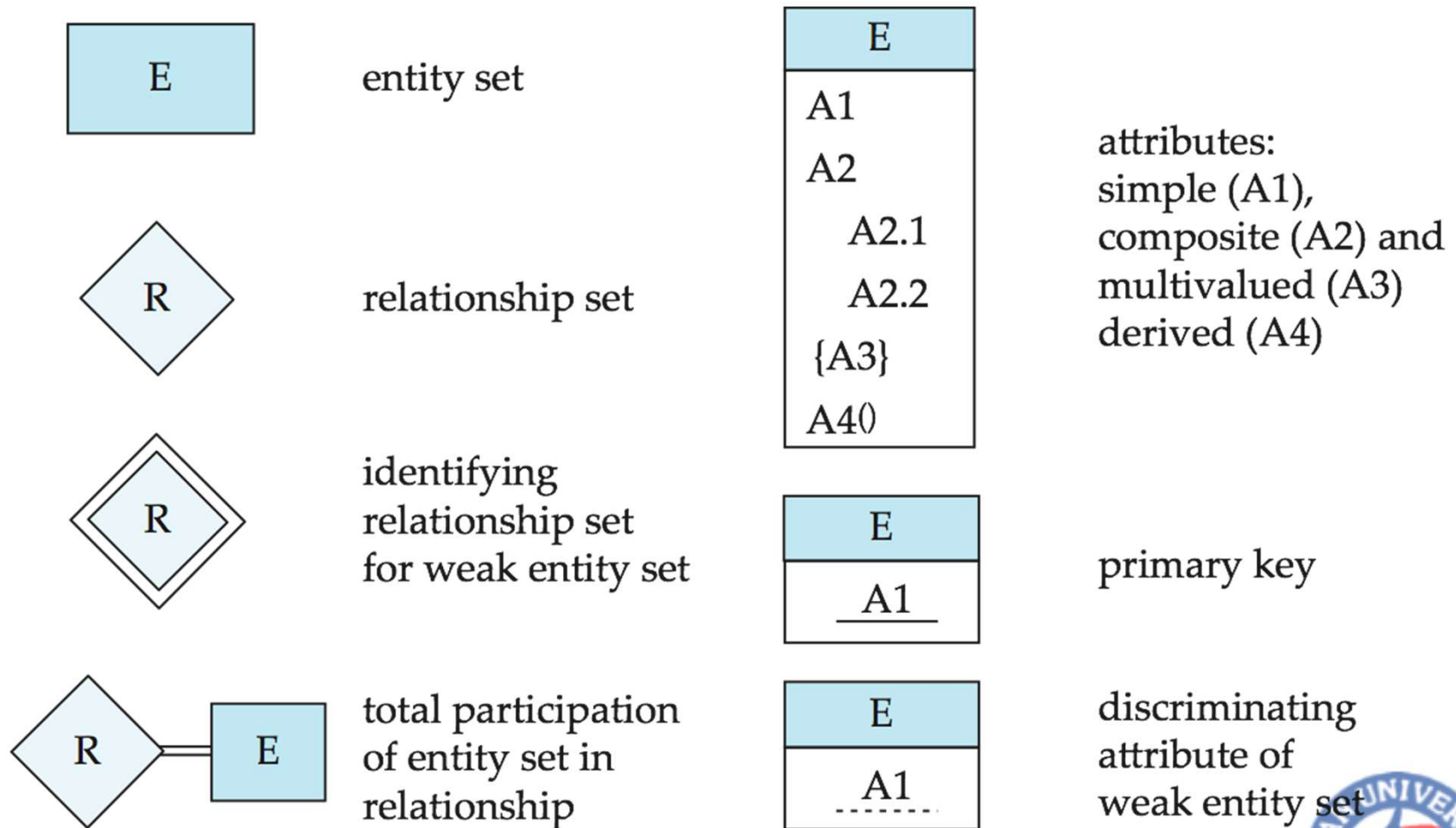


Summary of E-R Design Decisions

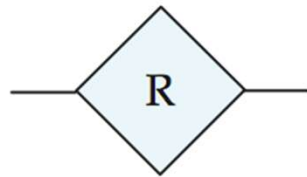
- ▶ The use of *an attribute or entity set* to represent *an object*.
- ▶ Whether a real-world concept is best expressed by *an entity set or a relationship set*.
- ▶ The use of *a ternary relationship* versus *a pair of binary relationships*.
- ▶ The use of a *strong or weak entity set*.
- ▶ The use of *specialization/generalization* – contributes to modularity in the design.
- ▶ The use of *aggregation* – can treat the aggregate entity set as a single unit without concern for the details of its internal structure.



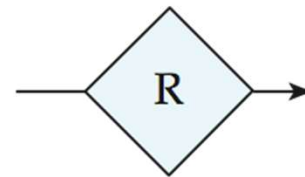
Summary of Symbols Used in E-R Notation



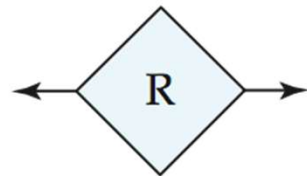
Symbols Used in E-R Notation (Cont.)



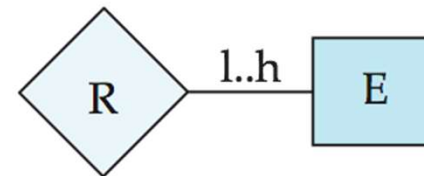
many-to-many
relationship



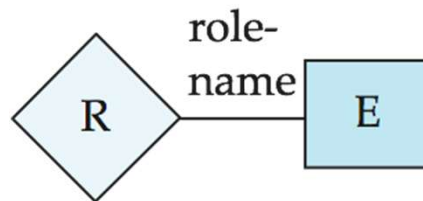
many-to-one
relationship



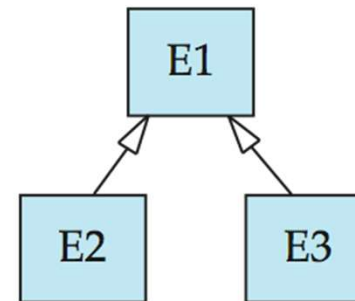
one-to-one
relationship



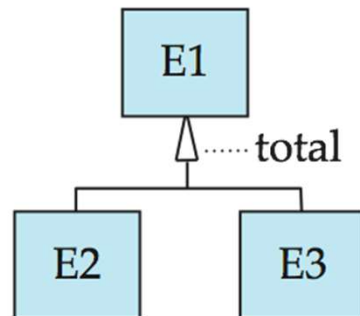
cardinality
limits



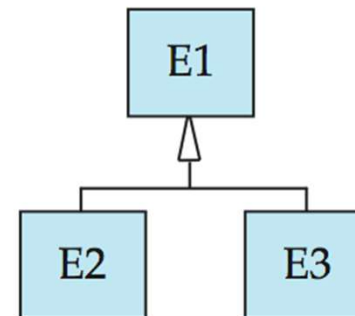
role indicator



ISA: generalization
or specialization



total (disjoint)
generalization

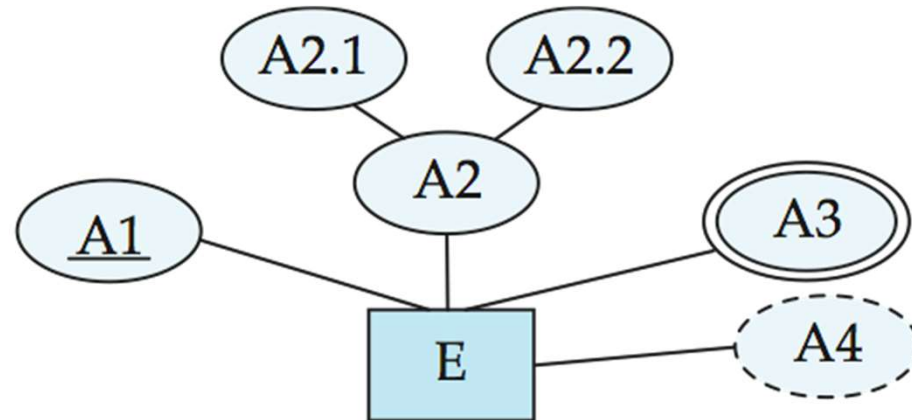


disjoint
generalization



Alternative E-R Notations

entity set E with
simple attribute A1,
composite attribute A2,
multivalued attribute A3,
derived attribute A4,
and primary key A1



weak entity set



generalization



total
generalization



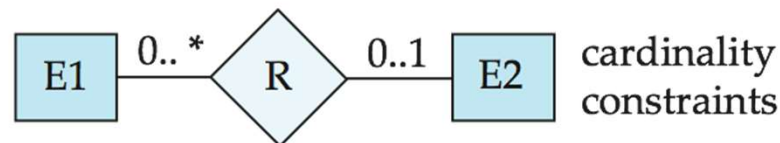
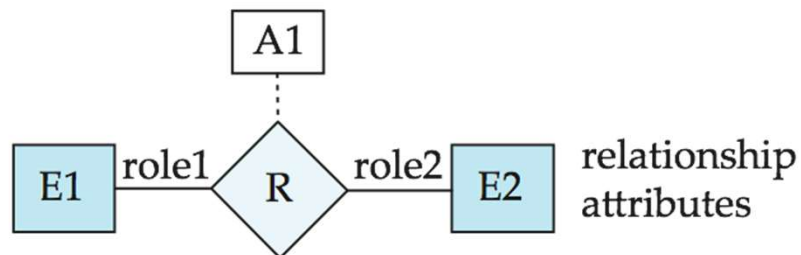
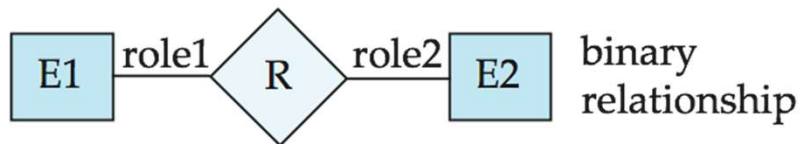
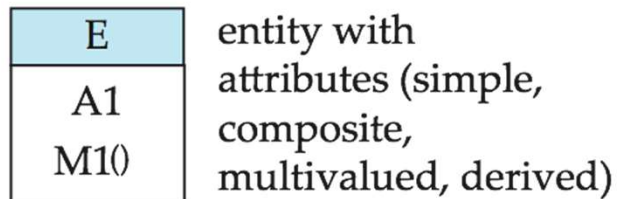
UML

- ▶ The **Unified Modeling Language** (UML) is a standard developed under the auspices of the **Object Management Group** (*OMG*) for *creating specifications of various components of a software system*.
- ▶ UML has many components to graphically model different aspects of an entire software system
- ▶ UML ***Class Diagrams*** correspond to ***E-R Diagram***, but several differences.

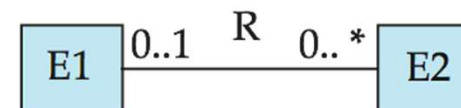
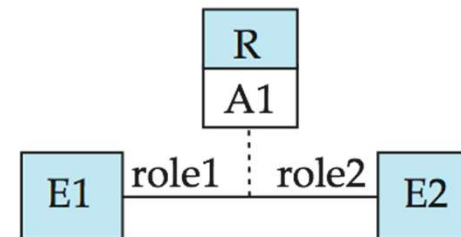
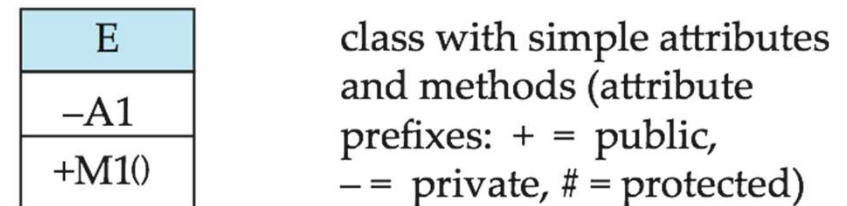


E-R vs. UML Class Diagrams

ER Diagram Notation

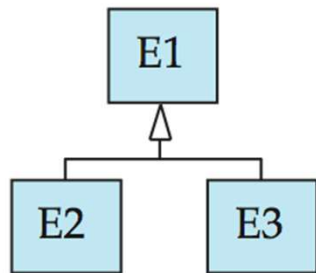
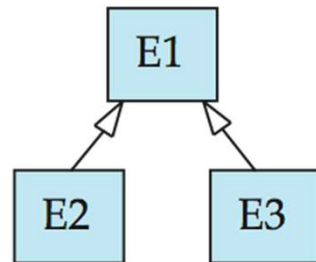
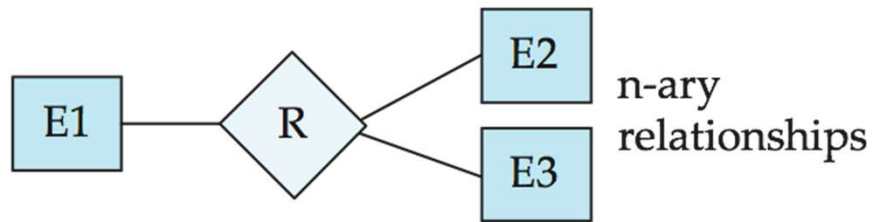


Equivalent in UML

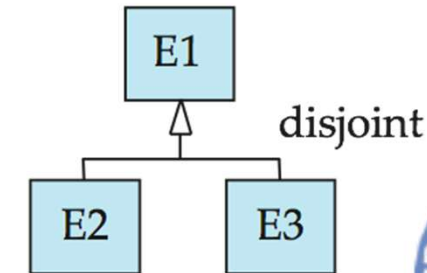
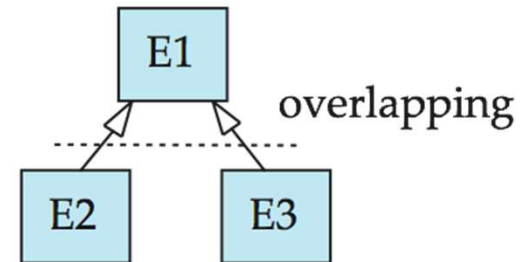
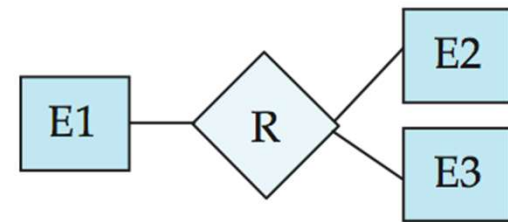


E-R vs. UML Class Diagrams

ER Diagram Notation



Equivalent in UML



Homework

- ▶ 7.20, and 题目1（非教材练习，按照PPT上的题目做）

- 7.20 Consider the E-R diagram in Figure 7.29, which models an online bookstore.
- List the entity sets and their primary keys.
 - Suppose the bookstore adds Blu-ray discs and downloadable video to its collection. The same item may be present in one or both formats, with differing prices. Extend the E-R diagram to model this addition, ignoring the effect on shopping baskets.
 - Now extend the E-R diagram, using generalization, to model the case where a shopping basket may contain any combination of books, Blu-ray discs, or downloadable video.



Homework

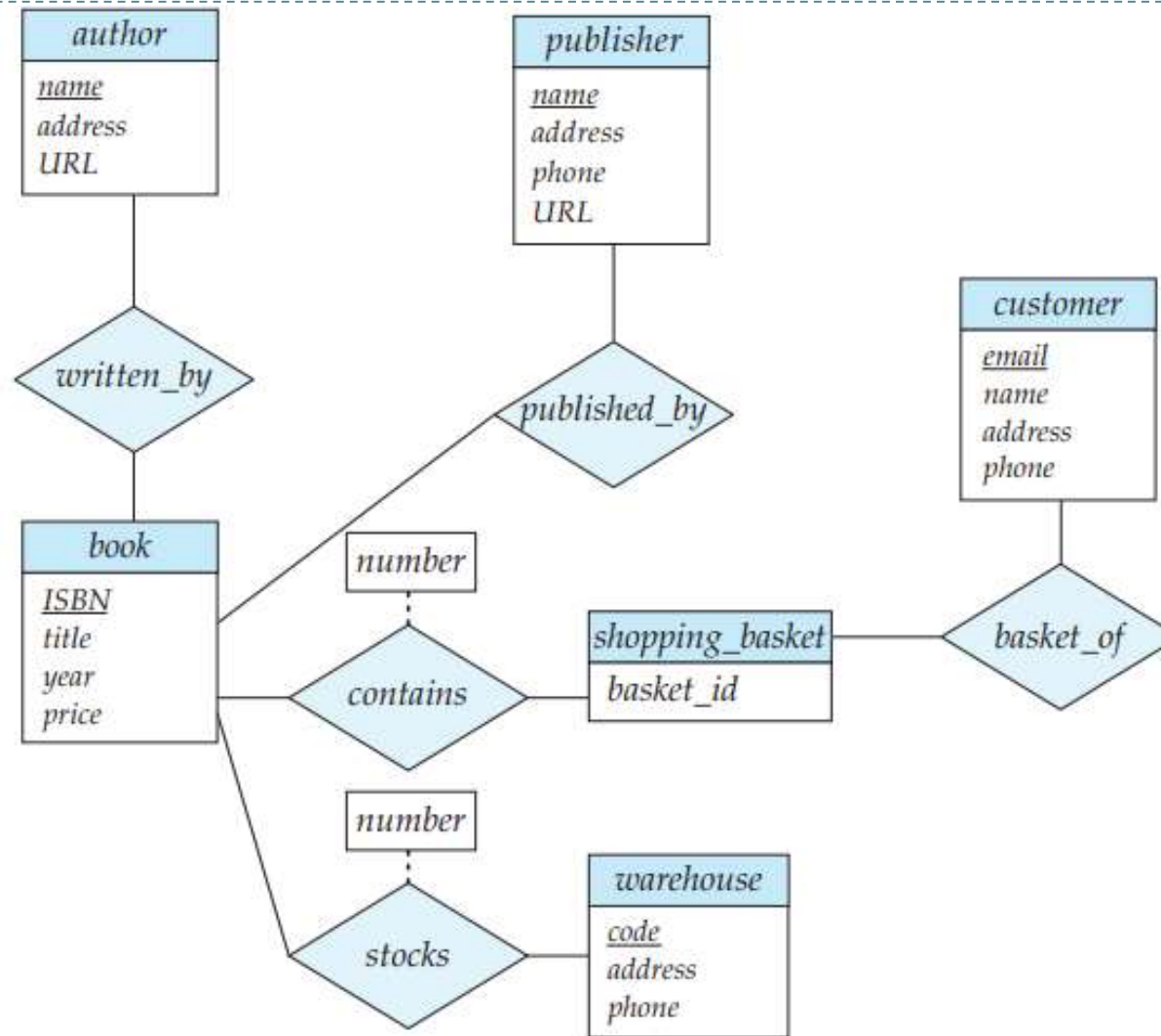


Figure 7.29 E-R diagram for Exercise 7.20.



Homework

▶ 题目1

- ▶ 医院住院部有若干科，每科有若干医生，病人住在病房中由某个医生负责治疗。每个医生只能属于一个科，每个病房也只能属于一个科。一个病房可住多个病人，一个病人由固定医生负责治疗，一个医生负责多个病人。为该医院的数据库设计E-R图，并列出关系模式的集合，以及包括主码约束。
- ▶ 各实体具有的描述属性如下，你可以根据情况自行添加或修改属性。
 - ▶ 科：科号、科名
 - ▶ 医生：医生编号、姓名、性别、级别
 - ▶ 病房：病房号、床位数
 - ▶ 病人：病人编号、姓名、性别

